

LAFC-Evict: A Large-Scale Counterfactual Benchmark for Learned Cache Eviction

Soroush Vahidi^{a,*}

^a*New Jersey Institute of Technology, Newark, NJ, USA*

Abstract

A single trace replay evicts one candidate per decision and so never reveals what would have happened under the alternatives – the counterfactual information needed to actually evaluate an eviction choice. This paper introduces LAFC-Evict, a reusable way to construct that counterfactual evidence at scale: for every resident candidate at every full-cache miss, a finite-horizon counterfactual miss-cost label, computed for real cache traces across five workload families and four cache capacities. The reusable release covers $H \in \{4, 8, 16\}$, and a validated extended sensitivity study compares $H \in \{16, 32, 64, 128\}$ without regenerating the canonical H16 labels. We evaluate this supervision surface against five questions. **(RQ1: is the target discriminative?)** Not uniformly: two-thirds of decisions are fully tied in aggregate and no audited decision has a strict unique winner, but discriminativeness varies strongly and predictably by workload, capacity, and horizon, and one workload is a fully degenerate, mechanistically explained negative control. **(RQ2: does the offline ranking correspond to closed-loop performance?)** We ran a closed-loop evaluation of LRU, MRU, random, and SIEVE across the five families at two capacities (20 random seeds). Offline and closed-loop policy orderings agree in most regimes where the offline signal is not near zero, with two characterized exceptions. **(RQ3: what drives this correspondence?)** Capacity-scale reuse (trace-derived LRU hit rate) correlates with both offline discriminativeness ($r = 0.83$) and closed-loop separation ($r = 0.90$), and discriminativeness and closed-loop separation correlate with each other ($r = 0.97$) (all $n = 10$, exploratory). The original

*Corresponding author.

Email address: `sv96@njit.edu` (Soroush Vahidi)

correspondence check is strongest at $H = 16$ within the release horizons; the extended $H \in \{16, 32, 64, 128\}$ sensitivity analysis shows that longer horizons generally add discrimination but that strong tie-degeneracy persists even at $H = 128$. **(RQ4: how sensitive are the labels to the continuation policy assumed after a forced eviction?)** A pre-registered sampled study (5,500 decisions, capacities 32 and 128) and a full-population MRU census (capacities 32, 64, 128, and 256) both find the canonical LRU-continuation labels robust to the tested alternatives. Random continuation is measurably more perturbing than MRU; we make no population-scale claim for random continuation. **(RQ5: what should this mean in practice?)** This offline supervision complements rather than replaces closed-loop evaluation, and we give concrete guidance for when it can be trusted. As a separate confirmatory check, we froze one model trained on this supervision and evaluated it in closed loop against matched LRU, MRU, SIEVE, LFU, and random baselines on a held-out protocol distinct from RQ2’s: it did not consistently outperform LRU, SIEVE, or random (1 win, 2 degenerate ties, 7 losses against each, on the same 10 cells), while consistently beating MRU and more often than not beating LFU – evidence that offline predictive structure on this target does not automatically translate into closed-loop gains for this frozen model. LAFC-Evict also releases the underlying candidate-, decision-, and pairwise-level supervision for reuse, with every quantitative claim traced to independently validated evidence.

Keywords: learned cache eviction, benchmark datasets, counterfactual supervision, cache replacement

1. Introduction

When a cache request misses and the cache is full, a system must pick one resident object to evict before admitting the requested object. Evaluating whether a given eviction decision was a good one requires knowing what would have happened to each candidate victim if it, rather than another object, had been evicted – information that is not directly observable from a single trace replay, because only one candidate is actually evicted at each decision. This paper studies a reusable way to construct that counterfactual evidence at scale, and evaluates when it is informative, when it degenerates, how it relates to actual closed-loop cache performance, and how sensitive it is to one of its own modeling assumptions.

L AFC-Evict materializes, for real cache traces, a per-decision, per-candidate finite-horizon counterfactual miss cost: for each object that could have been evicted at a full-cache miss, how many additional misses would occur over the next H requests if that object, specifically, had been the one evicted. This candidate-level quantity is stored as `y_loss` (a convenience alias `y_value = -y_loss` is also provided; see Section 8). Section 2 works through one full decision by hand – resident objects, a forced eviction, a future request window, and the resulting candidate costs, including a case where two candidates tie for optimal – before any aggregate statistic is introduced, so that every quantity used later in the paper is grounded in a concrete example first.

The contribution of this paper is not a new eviction policy. It is an evaluation of a reusable offline supervision surface against closed-loop cache-policy behavior, organized around five questions. **(RQ1)** Is the target discriminative between candidates, and when does it degenerate to an uninformative tie? Not uniformly: aggregate tie rates are substantial and no audited decision has a strict unique winner, but discriminativeness varies strongly and predictably by workload, capacity, and horizon, and one workload is a genuine, mechanistically explained negative control. **(RQ2)** Does the offline ranking correspond to actual closed-loop performance? In most workload/capacity regimes where the offline signal is not near zero, yes, with two specific, characterized exceptions. **(RQ3)** What drives that correspondence, and does it depend on horizon? Capacity-scale locality in the underlying trace correlates with both offline discriminativeness and closed-loop policy separation. Among the three horizons this paper evaluates ($H \in \{4, 8, 16\}$), $H = 16$ aligns offline and closed-loop evidence most strongly, though modestly; this paper also includes a separate long-horizon sensitivity study showing that the target remains substantially tied through $H = 128$. **(RQ4)** How sensitive are the labels to the continuation policy assumed after a forced eviction? Robust to the tested alternatives: a sampled study at two capacities finds the labels robust to MRU and mean-random continuation by a pre-registered criterion, and a full-population census extends the MRU result across all four evaluated capacities – arbitrary continuation policies remain outside this claim. **(RQ5)** What does this mean in practice for a researcher choosing between offline and closed-loop evaluation? The offline supervision this dataset provides complements rather than replaces closed-loop evaluation; we give concrete, evidenced guidance for when the offline signal can be trusted.

Every quantitative claim in this paper traces to a specific, independently validated evidence artifact; Sections 8 through 9 and the limitations discussion in Section 17 state the exact scope, sample sizes, and caveats attached to each result rather than presenting them as unqualified findings. The underlying release also exposes candidate-level, decision-level, and pairwise derived views of the same supervision at production scale (Table 1), which supports the evaluation methodology in this paper and is also intended for reuse by other researchers studying learned cache eviction; release construction, schema, and validation are described in Sections 3–7 but are not the paper’s central argument.

Section 18 positions this contribution precisely against learned-caching, benchmark, and offline-vs-deployed-evaluation literature; in brief, existing public cache-replacement resources do not provide a reusable candidate-level counterfactual supervision surface at this scale together with a direct evaluation of its relationship to closed-loop cache-policy behavior.

The rest of the paper proceeds as follows. Section 2 gives the worked example and motivates decision-aligned counterfactual supervision. Sections 3–7 describe construction, label definition, schema, and release validation. Section 8 defines the benchmark tasks. Section 9 reports target discriminativeness (RQ1) and, together with material drawn from the closed-loop production evaluation, addresses RQ2–RQ4 (offline/ closed-loop correspondence, workload/capacity/horizon dependence, and continuation-policy robustness); practical implications (RQ5) are synthesized alongside it. Section 15 then reports a separate, frozen learned-policy closed-loop check under its own held-out protocol. Section 17 states limitations, ethics, and reuse constraints, followed by related work and conclusion.

2. Background and a Worked Example

At each full-cache miss, a caching system must evict one resident object before admitting the requested one. Classical policies such as LRU, ARC [1], and LIRS [2] operationalize this choice through recency/frequency structure; more recent heuristics such as SIEVE [3] and S3-FIFO [4] revisit that structure for scan-resistance and simplicity. A recent hybrid line tunes such heuristics’ own parameters with a learned control plane rather than replacing them outright: LAH, instantiated as S4-FIFO over an S3-FIFO data plane, reports robustness and interpretability gains from this decoupling [5]. Learned or learning-augmented approaches – for example LeCaR, LRB, Par-

rot, and GL-Cache [6, 7, 8, 9] – instead attempt to predict which eviction will lead to lower future miss cost. CACHEUS, 3L-Cache, and HALP take the same general approach [10, 11, 12], typically training on labels each system derives for itself from raw traces. This makes cross-paper comparison hard: different papers use different traces, different label constructions, and different evaluation protocols, so a reported improvement is often not attributable to the policy alone.

LAFC-Evict addresses the label-construction half of that problem by materializing a reusable, candidate-level counterfactual label directly from real traces, independent of any specific learned method. The rest of this section works through exactly what that label is, using a small, fully hand-checkable example (full derivation in the paper’s reproducibility artifact); Section 3 then defines the same quantities formally for the production-scale dataset.

2.1. A worked eviction decision

Suppose a cache holds three objects at capacity 3, ordered from least-recently-used to most-recently-used: A, B, C. A request arrives for object D, which is not resident: a *cache miss*. The cache is full, so admitting D requires evicting exactly one resident object. A, B, and C are the three *candidate victims* for this *eviction decision*.

To score each candidate, we look ahead a finite *horizon* of $H = 3$ future requests, which turn out to be B, E, F. For each candidate, we ask: if this candidate, and no other, had been evicted, and the cache had then continued to evict under a *continuation policy* (LRU, the canonical choice throughout this dataset) for the next H requests, how many of those requests would miss? This defines the candidate’s *counterfactual miss cost*, stored as `y_loss` (a convenience alias `y_value = -y_loss` is also provided; see Section 3).

Figure 1 works through all three resulting counterfactual worlds by hand. Evicting A costs 2 future misses (B hits because A’s eviction did not affect B; E and F both miss because they are new objects). Evicting B costs 3 (B itself is now a miss, plus E and F). Evicting C also costs 2, by the same reasoning as A. **The optimal candidate set for this decision is $\{A, C\}$, a tie** – both evictions are equally good by this measure, and only evicting B is strictly worse. This is not a contrived edge case: across the full production dataset, no audited decision has a strict unique winner (Section 9), so tie handling of exactly this kind is the common case, not the exception.

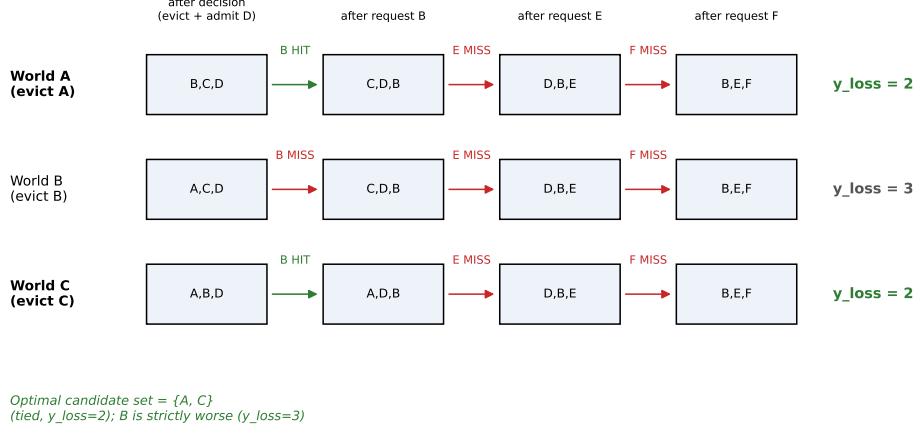


Figure 1: The worked example: three counterfactual worlds (one per candidate victim), each replayed forward under LRU continuation for the next $H = 3$ requests. A and C tie for optimal ($y_loss = 2$); evicting B is strictly worse ($y_loss = 3$).

2.2. From one decision to the released views

Grouping every candidate row by its decision id gives the *decision view* (candidate count, optimal-set membership, tie flags, regret summaries). Deriving pairwise preference labels directly from the candidate-level y_loss values – e.g. this decision yields (A,B): A preferred, (A,C): tie, (B,C): C preferred – gives the *pairwise derived view*. The pairwise view introduces no new ground truth: every label in it is a function of the candidate view already described. This is a narrower claim than systems such as HALP, which train directly on preference-comparison feedback as their primary learning signal [12]: LAFC-Evict’s pairwise view is instead a deterministic reformatting of the already-defined candidate-level y_loss values, released for compatibility with pairwise/ranking methods rather than as an independent source of supervision (Section 8 discusses why a derived pairwise view is still useful for certain modeling and diagnostic purposes). A single full-cache miss with k admissible candidates therefore appears in three aligned forms: k candidate rows, one decision-group summary, and up to $k(k - 1)/2$ within-decision pairwise comparisons.

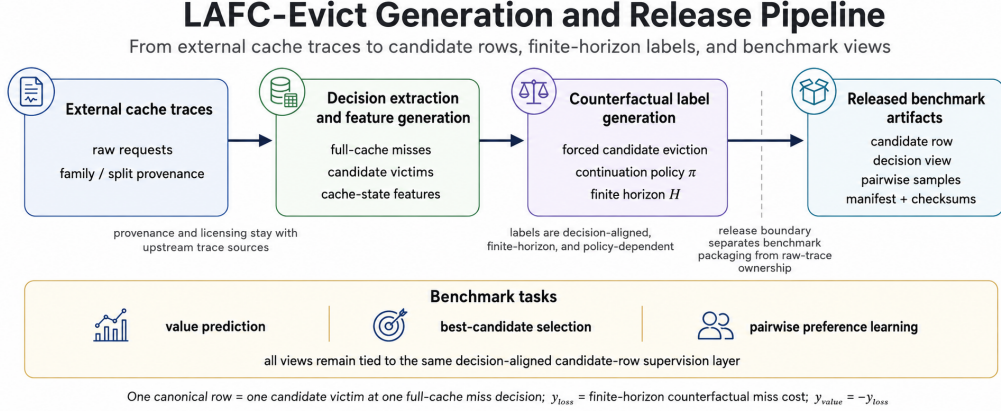


Figure 2: LAFC-Evict generation and release pipeline. External cache traces are converted into decision-level candidate rows, finite-horizon counterfactual labels, and released benchmark views for value prediction, best-candidate selection, and pairwise preference learning.

3. Evaluation Methodology: LAFC-Evict Construction

This section defines the scientific abstraction underlying LAFC-Evict before describing how it is packaged for release. A *trace* is a sequence of *requests* (accesses) to objects. A *cache state* is the set of objects resident in a bounded-capacity cache at a point in the trace. When a request misses and the cache is full, the system faces an *eviction decision*: choose one resident object, a *candidate victim*, to evict before admitting the requested object. LAFC-Evict treats every such decision, across every trace, capacity, and horizon evaluated, as one supervised unit; Section 8 defines the exact formal quantities (finite-horizon counterfactual loss, optimal candidate set, regret) built on top of this abstraction, following directly from the worked example in Section 2.

The cache abstraction used throughout this paper is intentionally narrow and stated explicitly here rather than left implicit: objects are unit-sized (no byte-weighted or variable-size accounting), capacity is counted in objects, and admission of the requested object is unconditional (eviction is never avoided by refusing admission). Section 17 discusses the consequences of this scope boundary; every claim in this paper should be read as bounded by it.

The evaluated dataset covers five trace families – `cloudphysics`¹, `metacdn`,

¹`cloudphysics` is a historical internal family identifier retained in project manifests,

`metakv`, `twemcache`, and `wiki2018` – chosen to span a CDN workload, a key-value workload, two additional production-derived workloads, and one intentionally included negative/control workload (`wiki2018`; Sections 9 and 12 show why it degenerates and treat that as a useful, explained result rather than a benchmark defect). Two further families, `brightkite` and `citibike`, are excluded from the open release subset for redistribution reasons; this is a release-packaging boundary, not a claim that the underlying generated corpus is limited to five families. Not every family appears in every split (e.g. `metacdn` contributes train/validation but no test decisions); split assignment is produced upstream of this release package and is normalized and validated, not assigned, here.

The evaluation in this paper is organized to satisfy four requirements:

1. **Decision alignment.** Preserve within-decision candidate competition rather than treating candidates as i.i.d. points.
2. **Task multiplicity.** Support pointwise (candidate-level), decision-level, and pairwise views of the same underlying supervision.
3. **Closed-loop grounding.** Evaluate the offline labels against actual closed-loop cache-policy behavior rather than presenting them as self-validating (Sections 10 and 11).
4. **Reproducibility.** Expose validated, checksummed evidence for every reported number (Section 7).

4. Experimental Methodology

This section consolidates the methodological choices used across every result reported in Sections 9–13 so that scope and caveats are stated once rather than scattered.

Workloads, capacities, horizons. All results use the five families defined in Section 3 (`alibaba-block`, `metacdn`, `metakv`, `twemcache`, `wiki2018`). Offline target-discriminateness results (Section 9) span all four evaluated

evidence records, and code for reproducibility. The underlying workload is Alibaba Cloud’s Elastic Block Storage production block-storage trace, not the VMware/CloudPhysics dataset. Reader-facing prose below refers to this workload as “`alibaba-block`.”

Table 1: Dataset scale from preserved-release metadata. Counts are manifest-backed or view-backed as noted.

metric	value
candidate rows	277,995,072
decision rows	2,363,286
pairwise-sample rows	1,000,000
preserved release size	2.7G
candidate parquet files	168
total release files	176
manifest file_inventory count	176
checksum entry count	175
selected trace families	alibaba-block, metacdn, metakv, twemcache, wiki2018
excluded trace families	brightkite, citibike
full validation status	passed

Table 2: Split counts from manifest-backed candidate-row metadata. These are candidate-row counts, not decision-view counts.

split	row_count
test	28,205,952
train	205,726,368
val	44,062,752

capacities (32, 64, 128, 256) and all three horizons ($H \in \{4, 8, 16\}$) in the reusable release. A separate validated long-horizon sensitivity experiment reuses canonical H16 and extends the same five-family, four-capacity characterization to $H \in \{32, 64, 128\}$; it is an analysis extension, not a change to the public release scope. Closed-loop, linkage, mechanistic, and continuation results (Sections 10–13) are reported at capacities 32 and 128 only – this is the scope of the Tier-1 production closed-loop evaluation on which all three later sections depend, not a data-availability limitation, and it is stated here once rather than repeated in each section.

Split terminology and two family-specific caveats. “test” denotes a temporally held-out scoring window; “validation” denotes a window used during earlier development that is not temporally held out in the same sense. Two caveats apply and recur throughout the paper: (i) **metacdn’s closed-loop evidence is validation-window evidence, not test-window**, for both of its evaluated capacities; every result that includes metacdn should

be read with this in mind, and Section 11 reports what changes if metacdn is excluded. (ii) **metakv is the only family whose scored window begins at an empty cache** (a cold start, identifiable from the identity $evictions = misses - capacity$ holding exactly for metakv and no other family); Section 12 discusses this as a plausible, not established, contributor to metakv’s one policy-ranking exception.

Policies and seeds. Closed-loop replay is evaluated for LRU, MRU, SIEVE (all deterministic, one run each), and random (20 CRN seeds, seeds 0–19; mean and per-seed standard deviation reported). SIEVE has no offline counterpart in the labels described in Section 5 and is therefore closed-loop-only evidence throughout – it is not part of any offline-vs-closed-loop correspondence claim, but is still reported as a closed-loop reference point (Section 10).

Offline metrics. Per decision: all-tied fraction, unique-winner fraction, mean/median optimal-set fraction, decision-weighted random-optimal probability, and random regret (mean and tail percentiles), all computed per (family, capacity, horizon) stratum before any pooling (Section 9).

Closed-loop metrics. Miss ratio per (family, capacity, policy), and, for random, its across-seed mean and standard deviation; relative miss-ratio difference vs. LRU is reported as the primary comparison statistic (Section 10).

Offline↔closed-loop linkage metrics. Sign concordance between the offline regret gap and the closed-loop miss-ratio gap (MRU-vs-LRU and random-vs-LRU only, since these are the two pairs with both an offline and a closed-loop side), plus Pearson/Spearman/Kendall correlation between the two gaps’ magnitudes, computed per horizon on the $n = 10$ (family, capacity) cells (Section 11). $n = 10$ is small and is treated as such throughout: correlations are reported with their exact n , never as population-scale claims.

Continuation-sensitivity metrics. Optimal-set Jaccard overlap, cross-continuation regret (in misses), and strict-reversal fraction, comparing the canonical LRU-continuation label against MRU- and mean-random-continuation (10 CRN seeds) relabelings of the same decisions, on a pre-registered, stratified 5,000-primary + 500-diagnostic decision sample at capacities 32 and 128, plus a full-population MRU census over all five families, capacities 32/64/128/256, and horizons 4/8/16 (Section 13). The primary statistical unit here is the *decision* (not the candidate row or the continuation seed): every reported sampled-study statistic is a decision-level cluster statistic, and the two reported bootstrap confidence intervals (one per continuation policy; Section 13) resample at the decision level; the MRU census is reported as a

complete population enumeration, not an inferential sample.

Tie-aware analysis. Every metric above that could be silently distorted by ties (optimal-set membership, pairwise preference, concordance) is computed with ties as a first-class outcome rather than broken arbitrarily; Section 2’s worked example shows the base case this generalizes from.

Provenance and validation strategy. Every number in this paper traces to a validated evidence record. The closed-loop, linkage, mechanistic, and continuation analyses were checked by reproducible scripts and, where applicable, by independent recomputation of headline values (Section 7). No number in this paper is read from a partial or unvalidated intermediate run; the full-population MRU continuation census is used only after complete-record validation and an independent recheck.

5. Counterfactual Label Generation

The benchmark label is defined on candidate victims rather than on completed policies. Let $r_1, r_2, \dots$ denote a request sequence, and let t index a request r_t that misses when the cache is full. Write S_t^- for the cache state immediately before the eviction choice; the candidate set $C_t = S_t^-$ is every resident object, so $|C_t|$ equals the cache capacity (32, 64, 128, or 256 objects in the evaluated release), counted in objects rather than bytes, with upstream object sizes not separately modeled. For any candidate $c \in C_t$, consider the intervention that evicts c , admits the missed request into the freed slot, and then follows a fixed continuation policy π for the next H requests, resolving every subsequent miss according to π ; in the evaluated release, π is LRU continuation rather than a Belady/offline-optimal rule. The resulting finite-horizon counterfactual loss counts those subsequent misses, so every miss contributes unit cost:

$$L_{t,H}^\pi(c) = \#\{\tau \in \{t+1, \dots, t+H\} : r_\tau \text{ misses in the induced trajectory}\}. \quad (1)$$

The release stores this quantity as `y_loss`, and it stores the corresponding value label as `y_value` = `-y_loss`. The triggering miss at time t is common to all candidates and is therefore not the decision-discriminating part of the label.

Both columns are retained in the schema, but they are not independent information: `y_value` is a deterministic sign-flip of `y_loss`, computed by the

same generation step. The two exist because downstream modeling code is written against either a minimization convention (`y_loss` as a cost/regret target, the convention used throughout the rest of this paper and the more directly interpretable of the two – a count of extra misses) or a maximization convention (`y_value` as a value/utility target, convenient for argmax-based candidate selection or value-function-style formulations). This paper treats `y_loss` as canonical and uses `y_value` only where a specific baseline is more naturally stated as a maximization, never reporting the same statistic under both names.

This label has two consequences for the paper. First, it creates a common numerical target for regression-style benchmark tasks. Second, it induces structured decision-level objects. For a fixed decision (t, S_t^-, C_t) , the benchmark-optimal candidate set is

$$C_t^* = \arg \min_{c \in C_t} L_{t,H}^\pi(c), \quad (2)$$

and the per-candidate regret is

$$\Delta_{t,H}^\pi(c) = L_{t,H}^\pi(c) - \min_{c' \in C_t} L_{t,H}^\pi(c'). \quad (3)$$

These quantities define best-candidate selection targets, pairwise preferences, and tie structure within a decision.

This supervision is a generated counterfactual label family, not an upstream annotation. It is finite-horizon, not asymptotic. It is tied to a continuation policy, not a universally optimal control objective. The label is therefore decision-aligned for the benchmarked eviction choice, but it is not an offline-optimal label in the sense of solving the full future control problem. These boundaries are central to the scientific positioning of the paper.

6. Dataset Schema and Views

The canonical candidate schema currently consists of decision metadata, candidate identity, split information, labels, and feature columns. Each row corresponds to exactly one candidate victim at exactly one full-cache miss *eviction event*, scored under one horizon. Trace identity and split metadata are recorded through `trace_name`, `trace_family`, `dataset_source`, and `split`. Cache configuration is recorded through `capacity` and `horizon`. `decision_id` identifies the underlying eviction event (trace, capacity, and

request position) and is shared by every horizon at which that same event is scored; `decision_t` gives its position in the request stream. Candidate identity is represented through the field `candidate_page_id`. The labels are `y_loss`, the finite-horizon counterfactual miss cost after forcing that candidate eviction under the fixed continuation policy, and `y_value` = $-y_loss$. Feature columns cover request context, candidate context, predictor/LRU signals, cache summary statistics, and recent request or hit rates.

Terminology used consistently for the rest of this paper: a *candidate row* is one row of the base schema above (one candidate at one eviction event, one horizon). A *decision* (interchangeably, *decision-horizon instance*) is the group of candidate rows sharing the same `decision_id` and `horizon` – since the same eviction event is independently scored at every evaluated horizon, one eviction event contributes one decision per horizon, not one decision overall. Reported decision counts (e.g. Table 1) are therefore decision-horizon counts, pooled across all evaluated capacities and horizons, not counts of distinct physical eviction events.

Beyond candidate rows, the release defines two derived views. The *decision view* aggregates all candidate rows that share a decision identifier and horizon and summarizes each resulting decision with candidate count, minimum loss, maximum value, the set of optimal candidate IDs, tie count, and regret summary statistics. The *pairwise view* compares candidate pairs from the same decision (same `decision_id` and horizon) and derives pairwise preference labels from the ordering of their `y_loss` values, with ties when the losses are equal. The evaluated open release does not ship the full pairwise view; it ships a capped pairwise sample of 1,000,000 rows instead.

Operationally, the schema can be read as a layered projection over the same eviction events: candidate rows are the atomic supervised objects, the decision view groups those rows by `decision_id` and horizon, and the shipped pairwise sample keeps only selected within-decision comparisons. Table 3 summarizes those column groups and derived fields directly in manuscript form.

7. Release Construction and Validation

The release pipeline is designed to package existing generated candidate rows into a benchmark artifact without requiring raw traces in the release package. The builder uses an out-of-core DuckDB path to ingest candidate

Table 3: Compact summary of candidate-row schema groups and derived-view fields.

Column group	Notes
Decision metadata	family, capacity, horizon, split, IDs
Candidate identity	candidate page ID
Labels	<code>y_loss</code> , <code>y_value</code>
Feature groups	request, candidate, cache, disagreement, rate
Derived decision fields	tie and regret summaries
Derived pairwise fields	A/B labels and regret differences

shards, normalize split values, emit partitioned Parquet candidate rows, derive the decision view, optionally create a capped pairwise sample, and write release metadata and checksums.

The validation surface is part of the benchmark contribution. The validation workflow includes:

- schema completeness for candidate partitions,
- split normalization and split-set consistency,
- presence of selected families and absence of blocked families,
- duplicate candidate detection within decision groups,
- consistency of decision metadata within a decision,
- decision-view schema and count consistency,
- checksum verification.

The evaluated open-release snapshot passed metadata repair, the provided verification suite, publication-bundle validation, and end-to-end release validation. The validation evidence comes from manifest-backed counts, schema checks, split checks, duplicate detection, and checksum verification implemented in reproducible scripts. The manuscript therefore reports validation of the release package without relying on unpublished hosting details. It does not claim that final public hosting or DOI assignment is complete. It also does not treat superseded status notes as the authoritative source of scientific evidence.

Table 4 summarizes the release-package components used in the evaluation: candidate rows, derived benchmark views, manifest metadata, and

Table 4: Release-package components for the evaluated benchmark snapshot. Locations are relative to the release root.

component	release location
candidate rows	data/candidate_rows/
decision view	data/decision_view/decision_view.parquet
pairwise sample	data/pairwise_sample/pairwise_sample.parquet
release manifest	metadata/release_manifest.json
checksums	metadata/checksums.sha256

checksums. Validation claims rely on those package contents together with reproducible validation scripts rather than on any standalone status note.

7.1. Provenance of the evaluation results in this paper

Beyond the released dataset above, every result reported in Sections 9–13 traces to an independently validated analysis record. These records cover target discriminativeness, Tier-1 closed-loop behavior, offline↔closed-loop linkage, mechanistic workload analysis, sampled continuation sensitivity, and the full-population MRU continuation census. Each record includes reproducible scripts and, where applicable, an independent recomputation of headline values. The manuscript reports only results whose validation and recheck steps completed.

Two label sources must not be conflated. The candidate-level `y_loss/y_value` labels released and evaluated throughout this paper use the canonical LRU continuation policy (Section 5). The continuation-sensitivity study (Section 13) additionally computes MRU- and random-continuation *relabelings* of sampled decisions and an MRU-continuation relabeling of the full census population, strictly for the purpose of measuring sensitivity – these sensitivity-study outputs are not an alternative release and are not used anywhere in this paper as if they were the canonical label.

The population-scale MRU continuation census is complete evidence. The census covers all 2,363,286 decision-horizon records in the evaluated corpus and passed an independent recheck before its summaries were imported into this manuscript. The raw 1.6 GB census output is preserved separately; the manuscript uses only compact summaries, validation reports, manifests, and Figure 7 source data.

A known, unresolved provenance limitation: the exact historical commit provenance of the full-release label generator is incomplete. This

does not affect the validity of the validated analyses above, each of which independently records its own generation context, but it means the full historical lineage of the generator code itself is not completely reconstructable from repository history alone.

8. Benchmark Tasks and Baselines

LAFC-Evict supports four benchmark tasks from one release.

Candidate-level value prediction.. Predict `y_loss` or `y_value` from a single candidate row. This is a pointwise task over the canonical benchmark unit and measures how well a model predicts finite-horizon counterfactual eviction quality for one candidate at one decision.

Best-candidate selection.. Given all candidate rows for one decision, identify a member of the minimum-loss set. This task can be framed as ranking or decision-level classification and connects most directly to the actual cache-eviction choice because it evaluates whether a method selects a benchmark-optimal victim within that decision.

Pairwise preference prediction.. Predict whether candidate A is better than candidate B within the same decision, where “better” means lower `y_loss` or equivalently higher `y_value`. This task uses the pairwise view or shipped pairwise sample as a simpler interface for within-decision preference modeling. Because equal losses induce ties, reports must state whether evaluation includes ties, excludes them, or treats them as a separate class. In the shipped sample, deterministic hash randomization of A/B orientation keeps the non-tie A-better/B-better split balanced (about 49.8%/50.2%) rather than encoding candidate-identifier order.

Candidate-label distribution characterization.. Summarize the empirical label surface without training a predictive model. This characterization task studies distributions of `y_loss`, `y_value`, regret, tie frequency, and decision ambiguity across families, capacities, horizons, and splits. It is descriptive rather than competitive, but it is important for understanding benchmark difficulty and for interpreting downstream task results.

The empirical scope of this paper remains intentionally narrow. Rather than presenting a large model bake-off, we report lightweight baselines that

can be executed directly from the committed release views and that establish reproducible reference points for later work.

The current results surface covers three such references. A linear regression baseline over candidate rows predicts `y_loss` from released features. It is intentionally weak: test MAE is 3.8739 with RMSE 4.7152 and $R^2 = 0.0026$, and the validation-split R^2 is slightly negative (-0.0506). Using that same linear score for within-decision selection nevertheless provides a reproducible decision-level floor. On the pairwise task, the earlier random and majority non-tie results remain useful sanity checks, but by themselves they do not test whether the released candidate features contain decision-relevant signal.

To test whether released candidate features carry pairwise signal without changing the benchmark schema, we join those features back to the capped shipped pairwise sample for evaluation only. This remains a sample-backed evaluation, not a full quadratic pairwise materialization. The canonical re-generated sample is itself tie-heavy: 878,262 of 1,000,000 rows (87.8%) are exact ties, leaving 121,738 non-ties. All classifier metrics below are therefore non-tie-only.

Fitting and evaluation protocol. Logistic score calibrations and the feature-difference model are fit only on non-tie `train` rows, with fixed L2 regularization and no validation tuning. The temporally held-out `test` split is the generalization result; train/validation columns are diagnostic. On the 2,211-row test non-tie subset, the logistic feature-difference model reaches 0.7078 accuracy and 0.5494 log loss, ahead of random/majority checks and the 1-D score baselines (Table 6). This replaces the earlier overall non-tie 0.9660 figure as the number to cite. The train/validation gap is large, and the test non-tie subset is small and unusually tie-heavy, so the result supports only a capped-sample, non-tie pairwise baseline, not a population or online-policy claim.

The best-candidate evaluator groups candidate rows by the canonical 9-column decision key and reports 2,363,286 decisions, an exact match with the decision view (Section 3). A cheap decision-level floor computed directly from the decision view shows why this match matters for interpretation: a uniform-random selector’s expected optimal-selection rate is 0.9912 with expected regret 0.0088, both more favorable than the linear-score selector’s observed 0.8753 optimal-selection rate and 0.1251 mean regret, purely because of the tie density documented in Section 9 – not because the linear score underperforms a stronger baseline. Headline selection-rate and regret numbers should therefore always be read alongside this random floor rather

Table 5: Pairwise-sample-backed summary statistics for the shipped sample view.

statistic	value
pairwise rows	1,000,000
split breakdown	test=102,024; train=737,896; val=160,080
family breakdown	wiki2018=250,872; alibaba-block=245,336; metakv=190,584; twemcache=169,064; metacdn=144,144
unique decisions represented	118,635
label distribution	a_better=60,673; b_better=61,065; tie=878,262
A/B orientation method	deterministic_hash_v1 (randomized, not lexicographic)

Table 6: Feature-based pairwise baselines on non-tie rows only (121,738 of 1,000,000 sampled pairs; 878,262 ties omitted). Models are fit on **train**; **test** is the primary held-out result.

baseline	feature / model	test ($n=2,211$)		train	val
		acc.	log loss	acc.	acc.
random non-tie	random-label check	0.4903	0.6931	0.5030	0.4971
majority non-tie	majority-class check	0.4853	0.6933	0.5030	0.5000
LRU-score pairwise	1-D logistic, LRU score	0.6427	0.9285	0.9141	0.9508
predictor-score pairwise	1-D logistic, predictor score	0.4853	0.6933	0.5030	0.5000
linear-score pairwise	logistic, linear score	0.6621	0.5512	0.9430	0.9735
logistic feature difference	logistic, feature diffs	0.7078	0.5494	0.9598	0.9896

than in isolation.

9. Target Discriminativeness (RQ1)

This section answers RQ1: is the finite-horizon counterfactual target discriminative between candidates, and when does it degenerate? Source: [analysis/sigmoid_target_discriminativeness_20260913/](#), audited over all 277,995,072 candidate rows / 2,363,286 decisions in the canonical dataset (all capacities, all horizons; the full population, not a sample).

Table 7: Benchmark task definitions and representative evaluation outputs.

Task	Inputs	Representative outputs
Value regression	candidate rows	MAE, RMSE, and optional rank correlation between predicted and observed values
Best-candidate selection	decision groups	Top-1 accuracy against the benchmark-optimal set and mean realized regret relative to the best available candidate
Pairwise preference	pairwise rows from one decision	Accuracy or ROC-AUC, with tie handling stated explicitly and non-tie-only reporting marked when used
Label characterization	candidate or decision aggregates	Histograms, quantiles, tie rates, regret summaries, and per-family/capacity/horizon breakdowns

9.1. The uncomfortable aggregate facts, stated first

Pooled across every family, capacity, and horizon: the decision-weighted random-optimal probability is 0.9912, the decision-weighted mean random regret is 0.0088, the all-tied fraction is 0.6766, and the unique-winner fraction is exactly 0.0. In plain terms: a uniformly random candidate choice lands in the optimal set 99.1% of the time, over two-thirds of decisions have every candidate exactly tied, and *no audited decision in this dataset has a single, strictly-best candidate victim*. The overall classification, by the audit’s own pre-registered rule, is **B = CONDITIONALLY DISCRIMINATIVE** – not uniformly discriminative, and this is stated here as the paper’s own finding rather than a limitation discovered post hoc. Aggregate degeneracy this large is, by itself, a substantive result about the target’s shape.

9.2. Where the target is informative

The aggregate picture, however, is not the same at every family, capacity, and horizon – consistent with prior large-scale production evidence that cache workloads vary substantially in access pattern and reuse structure even within a single deployment [13], which is why this section reports per-family results rather than only the pooled figure above. Figure 3 shows discriminativeness ($1 - \text{all-tied fraction}$) across all five families, four capacities, and three horizons (source data: same audit, per-stratum breakdown, `outputs/phase4_stratified_family_capacity_horizon.csv`, 60 family $\times$ capacity $\times$ horizon cells). Two families, **metacdn** and **twemcache**, are substantially more discriminative than the aggregate figure suggests – at $H = 16$, metacdn’s all-tied

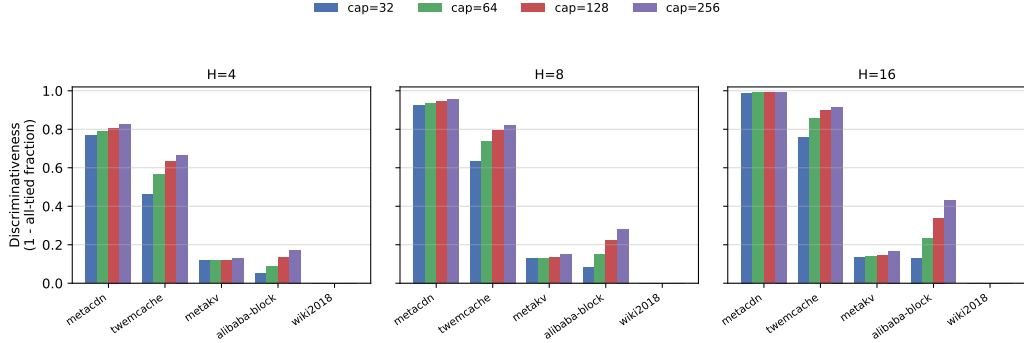


Figure 3: Target discriminativeness (1 - all-tied fraction) by family, capacity, and horizon. Reproducible from `paper/performance_evaluation/scripts/figures/make_figure2_discriminativeness.py` against the frozen audit CSV; no number is hand-entered.

fraction is 0.0085–0.012 and twemcache’s is 0.10–0.24 across its two smallest capacities, an order of magnitude below the aggregate 0.68. **alibaba-block** and **metakv** sit at the tie-heavy end (all-tied fraction 0.66–0.87 at $H = 16$), closer to but still below the degenerate extreme. **wiki2018** is exactly at the degenerate extreme: all-tied fraction 1.0 at every capacity and every horizon tested, with zero reuse events in its scored windows (Section 12 shows this is a mathematical certainty, not a statistical property of the trace, and treats wiki2018 as a genuine negative control rather than a benchmark failure). Discriminativeness also increases with horizon within every non-degenerate family (e.g. twemcache/cap32 rises from 0.46 at $H = 4$ to 0.76 at $H = 16$) – a longer look-ahead window resolves more ties, consistent with more future requests giving more opportunities for candidates’ fates to diverge.

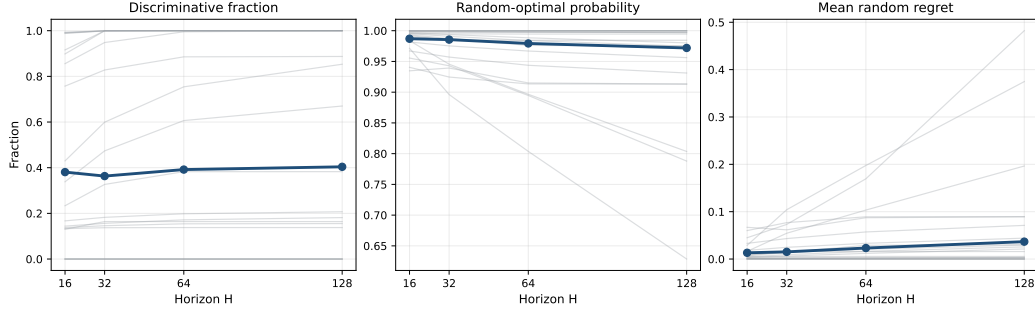


Figure 4: Extended horizon sensitivity for discriminativeness, random-optimal probability, and mean random regret. Thin gray lines are individual family $\times$ capacity cells; the dark line is the decision-micro aggregate.

Table 8: Extended horizon sensitivity for the offline target. Rows are decision-micro summaries over the five-family corpus. H16 reuses the canonical H16-eligible decision population; H32–H128 use the completed long-horizon production denominator, so paired cell-level deltas are used for within-cell interpretation.

Horizon	Decisions	All-tied	Discriminative	Random regret
16	787,762	0.6193	0.3807	0.0132
32	706,888	0.6367	0.3633	0.0152
64	706,888	0.6082	0.3918	0.0232
128	706,888	0.5961	0.4039	0.0367

The extended long-horizon comparison asks whether this picture changes when the look-ahead window grows beyond the release horizons. Table 8 and Figure 4 summarize the validated $H \in \{16, 32, 64, 128\}$ comparison. H16 is not regenerated: it is adapted from the canonical decision-view audit, while H32–H128 come from the completed Wulver long-horizon campaign. In paired family $\times$ capacity cells, the mean discriminative-fraction deltas are +0.0427 from H16 to H32, +0.0246 from H32 to H64, and +0.0095 from H64 to H128. Thus the changes diminish between H64 and H128, but a saturation claim would be too strong: effects remain heterogeneous by family and capacity. The target also remains strongly tie-degenerate at H128: the decision-micro all-tied fraction is 0.5961 and the cell-macro all-tied mean is 0.4681.

Table 9: Closed-loop policy evaluation: LRU miss ratio and relative miss-ratio difference vs. LRU for MRU, mean-random (20 seeds), and SIEVE. metacdn is validation-window evidence; all other families are test-window. Negative values mean the policy beats LRU.

family	cap.	LRU miss ratio	MRU Δ	random Δ	SIEVE Δ	random std
metacdn	32	0.5792	+47.37%	+5.38%	+0.81%	0.0013
metacdn	128	0.5490	+54.93%	+2.98%	+0.36%	0.0014
twemcache	32	0.7511	+27.12%	+4.19%	+8.47%	0.0025
twemcache	128	0.6205	+52.98%	+7.78%	+15.29%	0.0023
metakv	32	0.7410	+10.02%	+0.47%	+0.03%	0.0011
metakv	128	0.7290	+8.57%	-2.07%	-4.22%	0.0015
alibaba-block	32	0.9836	+1.65%	+0.28%	+0.20%	0.0010
alibaba-block	128	0.9561	+4.48%	+0.85%	+1.44%	0.0014
wiki2018	32	1.0000	0.00%	0.00%	0.00%	0.0000
wiki2018	128	1.0000	0.00%	0.00%	0.00%	0.0000

9.3. Interpretation

The scientific contribution here is partly *characterizing* where the target is informative rather than asserting it is informative everywhere. High tie rates are not hidden or explained away: they are the majority regime in this dataset, and Section 12 connects them directly to trace-level reuse structure (capacity-scale locality), giving a mechanistic account of exactly why metacdn/twemcache are discriminative and alibaba-block/metakv/wiki2018 are not. Sections 10 and 11 then ask whether this same discriminativeness structure predicts anything about actual closed-loop cache-policy behavior (it does, in the regimes identified here).

10. Closed-Loop Policy Evaluation (RQ2, part 1)

Section 9 characterizes the offline target. This section reports the actual closed-loop cache-policy behavior it will be compared against in Section 11 (Table 9). Scope: 5 families $\times$ 2 capacities (32, 128) $\times$ 4 policies (LRU, MRU, random-mean over 20 seeds, SIEVE [3]), per Section 4.

LRU is the best policy in 9 of 10 cells. The sole exception is metakv/cap128, where SIEVE (miss ratio 0.6982) beats LRU (0.7290) by 4.22% relative – the only cell in the entire evaluation where a non-LRU policy wins outright. **MRU is the worst policy in every non-degenerate cell**, by a wide margin (relative miss-ratio increase over LRU ranging from 1.65% at alibaba-block/cap32, a near-zero-signal regime, up to 54.93% at

metacdn/cap128). **Random-seed variability is small relative to the large policy gaps that drive this section’s headline comparisons:** the largest random-policy standard deviation observed is 0.0025 absolute miss ratio (twemcache/cap32), an order of magnitude smaller than twemcache’s own MRU gaps over LRU (27.12%/52.98% relative, ≈ 0.20 – 0.33 absolute miss ratio at twemcache’s LRU miss ratios). This margin does not hold uniformly, however: converted to the same absolute miss-ratio units, the smallest non-degenerate policy gap in the table (metakv/cap32 SIEVE, +0.03% relative on a 0.7410 LRU miss ratio, ≈ 0.0002 absolute) is smaller than the random-policy standard deviation itself, so in that low-signal cell the reported SIEVE/random differences are within, not above, the seed-noise floor and should not be over-read.

wiki2018 is a complete, exact tie across every policy: miss ratio 1.0 for LRU, MRU, SIEVE, and every one of random’s 20 seeds, at both capacities, with random’s standard deviation exactly 0.0. This is not approximate degeneracy – it is byte-for-byte identical across all 4 policies and 20 seeds. Section 12 shows this follows deductively from zero reuse events in the scored window, not as a statistical artifact of this particular closed-loop run.

alibaba-block is a near-tie regime at both capacities: every policy’s relative gap to LRU stays under 4.5%, an order of magnitude smaller than metacdn’s or twemcache’s gaps. **The metakv/cap128 cell is the one genuine exception to ”LRU dominates”,** and there both SIEVE (miss ratio 0.6982) and random-mean (approximately 0.7137, relative gap -2.07%) beat LRU (0.7290), while MRU remains worst; Section 12 investigates, but does not fully resolve, why. metacdn’s results should be read as validation-window evidence throughout (Section 4); twemcache’s are test-window.

No universal middle-of-the-pack ranking is claimed for SIEVE vs. random: SIEVE beats random-mean at metacdn and metakv/cap32, but random-mean beats SIEVE at twemcache (both capacities), alibaba-block (both capacities), and metakv/cap128 – the relative order of these two policies is family-dependent, not fixed. Only one ordering statement holds across every non-degenerate cell: **MRU is worst everywhere.** LRU is best in 9 of 10 cells and falls only as low as third place, at metakv/cap128, in the tenth.

11. Offline $\leftrightarrow$ Closed-Loop Correspondence (RQ2, part 2)

This section answers the second half of RQ2 – does the offline candidate ranking from Section 9 correspond to the closed-loop behavior in Section 10? – and, jointly with the horizon comparison below, RQ3’s horizon question. Source: `analysis/closed_loop_offline_linkage_20260914/`, computed with two independent code paths in numerical agreement. **SIEVE has no offline counterpart** (Section 5 defines labels only for the continuation-policy family LRU/MRU/random) and is therefore excluded from every comparison in this section; only MRU-vs-LRU and random-vs-LRU are compared.

RQ-CL1 (rank agreement). At $H = 16$, across the 10 family $\times$ capacity cells: MRU-vs-LRU agrees in sign (offline regret-gap sign matches closed-loop miss-ratio-gap sign) in 7 of 10 cells, disagrees in 1, and is tied on both sides (wiki2018) in 2; random-vs-LRU agrees in 6, disagrees in 2, ties in 2. The discordant cells are always the same two: **alibaba-block/cap32** (discordant for both pairs – offline regret gaps there are -0.0027 and -0.0007 , i.e. barely non-zero on the offline side too) and **metakv/cap128** (discordant for random-vs-LRU only – offline says random is very slightly worse than LRU, $+0.0013$ regret gap, while closed-loop shows random modestly beating LRU, -0.015 miss-ratio gap; Section 10 additionally notes SIEVE also beats LRU there). Excluding wiki2018’s two trivially-tied cells, concordance among the 8 non-degenerate cells is not manufactured by wiki2018’s automatic agreement: dropping wiki2018 still leaves the same two discordant cells and the same qualitative picture.

RQ-CL2 (gap-magnitude correspondence). At $H = 16$: MRU-vs-LRU Pearson $r = 0.835$ (Spearman $\rho = 0.915$, Kendall $\tau = 0.773$); random-vs-LRU $r = 0.755$ ($\rho = 0.829$, $\tau = 0.591$). Figure 5 shows both relationships directly, with the two discordant cells annotated rather than hidden. These correlations rest on only 8–10 independent (family, capacity) points and are reported with that n attached everywhere; they are not treated as population-scale estimates.

RQ-CL3 (workload/capacity dependence). `metacdn` and `twem-cache` are the strong-agreement, high-discriminateness regimes and also show the largest closed-loop policy separation; `alibaba-block` has a near-zero offline gap whose sign is not robust; `metakv/cap128` is the one reversal; `wiki2018` is degenerate on both sides. Across the 10 cells at $H = 16$, an exploratory correlation between offline discriminativeness ($1 - \text{all-tied fraction}$)

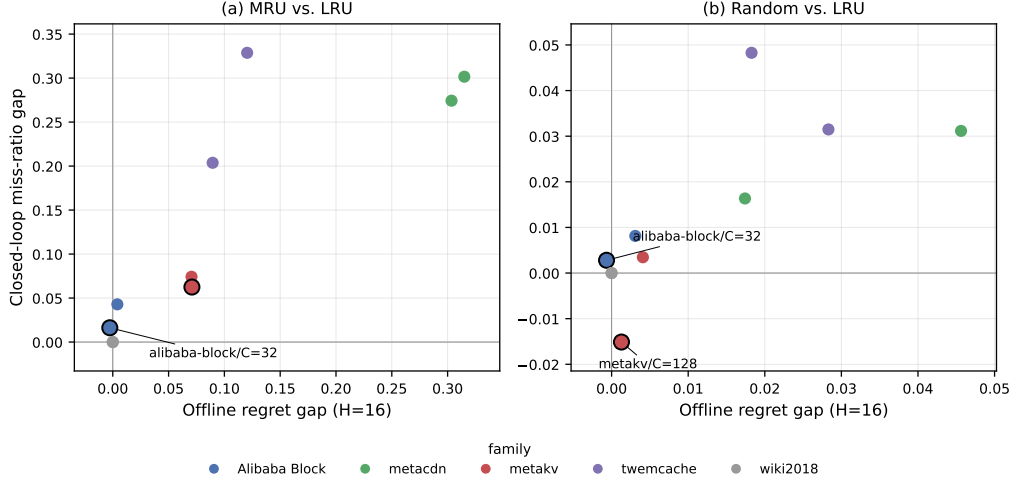


Figure 5: Offline regret gap vs. closed-loop miss-ratio gap at $H = 16$, split into two panels: (a) MRU-vs-LRU and (b) random-vs-LRU, each $n = 10$ family $\times$ capacity cells (one point per cell per panel). **The two panels use different axis ranges**, visible directly from their tick labels: random-vs-LRU gaps (panel b) are uniformly much smaller in magnitude than MRU-vs-LRU gaps (panel a), so each panel is scaled to its own range rather than a shared one. The two discordant cells from RQ-CL1 are outlined and labeled; every other point is left unlabeled. Reproducible from `paper/performance_evaluation/scripts/figures/make_figure3_linkage_scatter.py` against `outputs/rq_cl2_scatter_data.csv`.

and closed-loop $|\text{MRU} - \text{LRU}|$ separation gives Pearson $r = 0.968$; Section 12 traces this to shared trace locality.

RQ-CL4 (horizon). $H = 16$ wins or ties on every magnitude-sensitive statistic across every sensitivity slice tested (primary, excluding wiki2018, test-window-only), for both pairs. The coarse concordant/ discordant/tied counts are *identical* at $H = 4$, $H = 8$, and $H = 16$ (the same two cells are discordant regardless of horizon); horizon changes the *strength* of the correspondence (the correlation coefficients above), not which cells agree in sign. The differences between horizons are modest in absolute terms ($r = 0.746$ at $H = 4$ to $r = 0.835$ at $H = 16$ for MRU-vs-LRU) and $n = 8$ –10 per horizon is small, but the pattern is consistent and not cherry-picked.

Answer to RQ2/RQ3. Offline and closed-loop policy orderings agree in the large majority of workload/capacity regimes where the offline signal itself is not near zero, with two specific, characterized exceptions; $H = 16$

gives the strongest, though modest, alignment among the three horizons tested (Section 17 states the horizon scope explicitly: $H \in \{4, 8, 16\}$ only, no claim beyond $H = 16$). This is a bounded, evidenced claim, not a universal predictive-validity claim for the offline labels.

12. Mechanistic Workload Analysis (RQ3, part 2)

Sections 9 and 11 show *that* discriminativeness and offline $\leftrightarrow$ closed-loop agreement vary by workload; this section asks *why*, using only existing trace-only and closed-loop evidence (no new simulation). Classifications follow the source report’s own discipline exactly: DIRECTLY OBSERVED (a deductive or exactly-reproduced fact), CONSISTENT WITH MECHANISM (plausible, structurally supported, not proven), and CAUSALLY ESTABLISHED (reserved for one deductive result below). No claim here is upgraded beyond its source classification.

Capacity-scale reuse correlates with both discriminativeness and closed-loop separation. Trace-derived LRU hit rate (an exact, capacity-scale reuse measure, computed independently of the labels) correlates with offline discriminativeness at $r = 0.83$ and with closed-loop $|\text{MRU} - \text{LRU}|$ separation at $r = 0.90$ (pooled $n = 10$). Figure 6 shows both relationships directly. This is an $n = 10$, exploratory finding, consistent with but not independent evidence from RQ-CL3’s own exploratory correlation (Section 11) – the same underlying trace property is driving both.

alibaba-block/cap32 is near-zero signal. Only 17.5% of window requests are reuses at all; of those, only 9.4% fall within stack distance 32. The trace-only predicted LRU hit rate ($0.175 \times 0.094 = 1.6\%$) exactly reproduces the observed Tier-1 LRU hit rate ($134/8192 = 1.64\%$). The offline regret gaps there ($-0.0027, -0.0007$) are tiny on a scale where metacdn’s own MRU-LRU gap is over $100\times$ larger. The Section 11 discordance at this cell is exactly what a coin-flip-scale signal would produce.

metakv/cap128 remains a bounded exception. metakv’s reuse-distance median is short (2, comparable to metacdn’s 3) but its tail is markedly heavier: reuse-within-32 is only 0.574, rising to just 0.600 by distance 128 – a narrow, 2.6-percentage-point “boundary zone” sitting almost exactly at the tested capacities, where a recency-strict policy (LRU) and a second-chance policy (SIEVE) can plausibly diverge. metakv’s cold-start scoring window (Section 4) is a second, independently real structural property of this same cell. **Neither is shown to be the cause:** no per-request

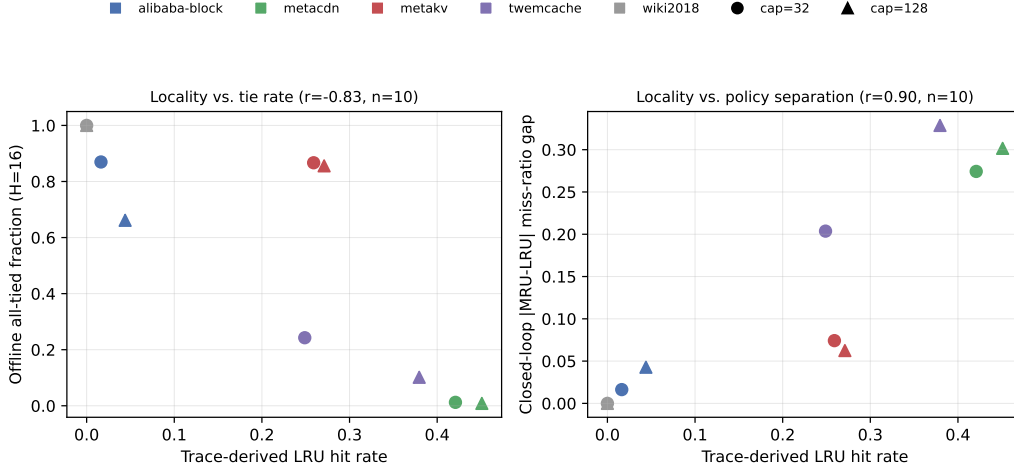


Figure 6: Trace-derived LRU hit rate vs. offline discriminativeness (left) and closed-loop policy separation (right), $n = 10$ family $\times$ capacity cells. Reproducible from `paper/performance_evaluation/scripts/figures/make_figure4_locality.py` against `outputs/mechanism_comparison.csv`.

SIEVE or random trajectory log exists in the validated evidence to isolate which property (or their interaction, or neither) produced the reversal, and the source analysis states this explicitly rather than forcing a resolution.

twemcache’s random-over-SIEVE ordering is not explained by existing data. twemcache has the strongest “small hot set plus long tail” bi-modal signature in the dataset (longest novel streak 3; reuse distance $p_{10}=3$ but $p_{75}=7379$), which would naively predict SIEVE’s second-chance admission should help – but the observed result is the opposite (SIEVE loses to random at both capacities). Existing trace-only data can describe this structure but cannot explain the direction of the outcome; the source analysis classifies any “SIEVE benefits from bimodality” story here as NOT SUPPORTED by the data, and this paper does not repeat that story.

wiki2018 (causally established): the one exception to the paper’s general classification discipline, and deliberately so. Zero reuse events occur in the scored window ($n_{\text{reuse}} = 0$ across all 4096 requests, every one a first-ever occurrence over the full 50,000-request trace). Since a hit requires a repeated reference to an already-resident item, miss ratio = 1.0 for every policy is a mathematical certainty given zero reuse events, not a

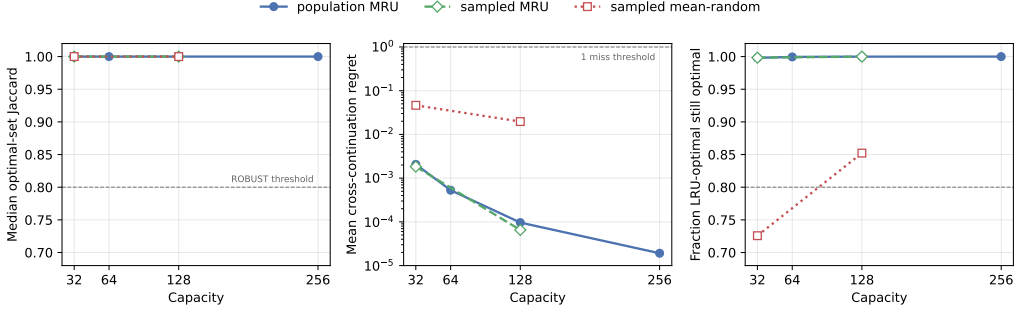


Figure 7: Continuation-policy sensitivity in the pre-registered informative stratum (Set C: decisions discriminative under LRU). Sampled MRU and mean-random evidence covers capacities 32 and 128 only; full-population MRU evidence covers capacities 32, 64, 128, and 256. No population-scale random-continuation result is shown or implied. The left panel plots *median* Jaccard against the 0.8 threshold because the pre-registered Set-C rule is defined on median Jaccard, not the mean; the center panel plots mean cross-continuation regret for descriptive scale against the raw 1-miss bound, but the rule itself applies that bound to the *fraction* of LRU-optimal candidates remaining optimal (right panel, reported in text), not to the mean regret directly.

statistical tendency – an existence proof from a directly counted quantity, which is why this one result is classified CAUSALLY ESTABLISHED while every other mechanism in this section is not.

13. Continuation-Policy Sensitivity (RQ4)

The canonical labels (Section 5) use LRU as the continuation policy after a forced eviction. This choice matters because a deployed learned policy would control later evictions, not necessarily LRU. RQ4 therefore asks whether the optimal-candidate set and regret ordering induced by the canonical labels are materially changed when the same forced eviction is followed by a different continuation policy. Table 10 summarizes the linkage and continuation-sensitivity evidence referenced throughout this section.

Sampled MRU/random study. The pre-registered sampled study evaluates 5,000 primary decisions plus 500 diagnostic oversample decisions at capacities 32 and 128. The implementation rechecked the LRU baseline labels exactly before evaluating alternative continuation policies. By the pre-registered Set-C rule (median Jaccard ≥ 0.8 and cross-continuation regret $\leq$

1 miss for $\geq 80\%$ of LRU-optimal candidates), both MRU and mean-random continuation classify ROBUST at this sampled scope. MRU is very close to LRU: Set-C mean Jaccard is 0.9958 at capacity 32 and 0.9999 at capacity 128, with mean cross-continuation regret 0.0018 and 0.0001 misses, respectively. Mean-random continuation also classifies ROBUST, but it is visibly more perturbing: mean Jaccard is 0.7251 at capacity 32 and 0.8522 at capacity 128, with mean cross-continuation regret 0.0463 and 0.0197. A pre-registered bootstrap (2,000 resamples, $n = 6,597$ pooled Set-C decisions) gives 95% confidence intervals for mean cross-continuation regret of $[0.00073, 0.00103]$ misses for MRU and $[0.03049, 0.03337]$ misses for mean-random, both well clear of zero and consistent with the point estimates above.

Full-population MRU census. The full-population MRU census extends the MRU side of that result to all five families, capacities 32, 64, 128, and 256, and horizons 4, 8, and 16, over the complete population of 2,363,286 decision-horizon records. Validation found no duplicate, missing, or extra decision keys, and an independent recheck passed before these summaries were used. Across all decision-horizon units, mean optimal-set Jaccard is 0.9994054593, median Jaccard is 1.0, 98.69% of records have exact Jaccard 1, mean cross-continuation regret is 0.000304471 misses, median and p95 regret are both 0, the maximum mean regret is 0.193548 misses, and 99.9708% of LRU-optimal candidates remain MRU-optimal on average. Strict preference reversals are rare: 4,646 discordant candidate pairs out of 24,791,643,360 pairwise comparisons, a fraction of 1.87×10^{-7} .

Informative-stratum result. Aggregate numbers alone would be misleading because both-tied decisions are common: 67.31% of all decision-horizon records are all-tied under both LRU and MRU. The pre-registered decisive stratum is therefore Set C, excluding decisions already all-tied under LRU. In this informative stratum, the census has 764,282 decision-horizon units from 299,905 eviction decisions. Set-C mean Jaccard is 0.9984587309, median Jaccard is 1.0, the strict reversal fraction is 5.38×10^{-7} , mean cross-continuation regret is 0.000638213 misses, and the mean fraction of LRU-optimal candidates still MRU-optimal is 0.9993955215. Since wiki2018 contributes no Set-C records, the Set-C result is not inflated by the degenerate negative control; the all-family and excluding-wiki Set-C summaries are identical.

Capacity and horizon effects. Sensitivity is largest at the smallest capacity and longest horizon, but remains small in absolute terms. In Set C, mean Jaccard/mean cross-continuation regret are 0.994809/0.002067

at capacity 32, 0.998859/0.000530 at 64, 0.999806/0.000096 at 128, and 0.999959/0.000019 at 256. The added population-only capacities therefore reveal no hidden sensitivity. By horizon, mean Jaccard declines from 0.999820 at $H = 4$ to 0.999222 at $H = 8$ and 0.996869 at $H = 16$, while mean cross-continuation regret rises from 0.000039 to 0.000283 and 0.001355 misses. The direction is real, but the magnitude remains small.

Worst cell and scope. The worst Set-C family/capacity/horizon cell is alibaba-block/capacity 32/ $H = 16$: mean Jaccard 0.980433, median Jaccard 1.0, mean cross-continuation regret 0.003899 misses, and strict reversal fraction 3.45×10^{-5} . Even this cell satisfies the pre-registered robustness criterion. These results address the reviewer concern within the evaluated scope: sampled MRU and random at capacities 32/128, and full-population MRU at capacities 32/64/128/256. They do not establish population-scale random-continuation robustness, and they do not show that every possible deployed continuation policy would preserve the same ordering.

14. Practical Implications (RQ5)

This section synthesizes Sections 9–13 into guidance, without turning exploratory correlations into universal rules.

When should the offline counterfactual signal be trusted? Most directly in workload/capacity regimes with high capacity-scale reuse – identifiable, before running any closed-loop replay, from a low trace-derived all-tied fraction or a high trace-derived LRU hit rate (Section 12). In this dataset that means metacdn- and twemcache-like regimes, where offline discriminativeness is high and offline $\leftrightarrow$ closed-loop agreement is strongest.

When should a researcher be cautious? In near-zero-offline-signal regimes (alibaba-block/cap32 is the concrete example here): a small offline regret gap can flip sign without indicating a real disagreement – it is consistent with, not evidence against, offline informativeness elsewhere. Caution is also warranted around cold-start-scored windows (metakv) and validation-window-only evidence (metacdn), both stated explicitly in Section 4 rather than left implicit.

What does a high tie rate mean? Not that the target is useless – Section 9’s aggregate all-tied fraction of 0.68 coexists with genuinely discriminative sub-regimes. A high tie rate is itself informative: it identifies where a policy comparison is unlikely to be resolvable by this label family at all

(wiki2018 is the limiting case), which is useful information for deciding where *not* to expect a learned method to outperform a heuristic on this evidence.

Why still run closed-loop evaluation? The offline/closed-loop association in Section 11 is real but bounded: it holds in most, not all, regimes, rests on $n = 8\text{--}10$ cells, and has at least one unexplained exception (metakv/cap128). Offline-label results therefore cannot be presented as closed-loop performance claims.

How should continuation-policy assumptions be interpreted? As a tested, not assumed, robustness property, and one with real limits: the labels are close to unaffected by switching to MRU continuation at population scale across capacities 32/64/128/256, more (though still boundedly) affected by random continuation in the validated sample at capacities 32/128, and the sampled random effect grows somewhat with horizon (Section 13). This supports using the canonical LRU-continuation labels with reasonable confidence in the evaluated regimes, not a claim that continuation choice is irrelevant.

What capacity-scale reuse tells you before running anything. Trace-derived LRU hit rate at the target capacity is the statistic most strongly associated with both offline informativeness and closed-loop policy separation in this $n = 8\text{--}10$ exploratory evidence. It is therefore a useful screening signal for where to look next, not a guarantee of what a new trace or deployed policy will do.

15. A Frozen Learned-Policy Closed-Loop Check

Section 10 evaluates non-learned policies against the Tier-1 production closed-loop protocol. This section reports a separate, smaller confirmatory experiment: freezing one model trained on this benchmark’s supervision and evaluating it, in closed loop, against matched baselines under its own held-out test protocol. **This experiment uses an independently constructed held-out test split (the frozen model’s own training-pipeline split, applied identically across all five families, including metacdn) and is not run on the Tier-1 protocol; its miss-ratio values are therefore not numerically comparable, cell for cell, to Table 9.** What is comparable, and is the point of this section, is the ordering among policies evaluated together under one matched, held-out protocol.

Setup. A single HistGradientBoostingRegressor was trained to predict the canonical $H = 16$ `y_loss` target, selected by an offline validation-set

regret/MAE criterion before any closed-loop replay was run, and frozen (no retraining, retuning, feature change, or post-hoc adjustment at any later point). It was evaluated by closed-loop replay – evict the resident candidate with the lowest predicted `y_loss` at every full-cache miss – on the same 10 predetermined family $\times$ capacity cells (5 families $\times$ capacities 32, 128) used elsewhere in this paper, each scored on a fixed, pre-registered held-out test interval, against matched LRU, MRU, SIEVE, LFU (each one deterministic run) and random (20 seeds) baselines under identical warmup (empty cache and empty bounded-history state at the test boundary) and scoring conventions.

Table 11: Frozen learned-policy closed-loop check: win/tie/loss count for the learned policy against each comparator across the same 10 predetermined cells (5 families $\times$ capacities 32, 128), under a held-out protocol distinct from Table 9. A tie is exact floating-point equality; every observed tie is one of the two wiki2018 cells, where all 6 policies score miss ratio 1.0. Macro miss ratio is the unweighted mean over the 10 cells (lower is better).

comparator	learned wins	ties	learned losses	macro miss ratio (comparator)
LRU	1	2	7	0.7117
MRU	8	2	0	0.8484
SIEVE	1	2	7	0.7096
LFU	5	2	3	0.7825
random (20-seed mean)	1	2	7	0.7228
learned macro miss ratio				0.7629

Result. Table 11 reports the outcome the frozen policy did *not* achieve: outside the two fully degenerate wiki2018 cells (miss ratio 1.0 for every policy, as in Table 9), the learned policy beat LRU, SIEVE, and the random-seed mean in exactly one of the 10 cells each – the same cell, metakv/cap128, in all three comparisons, by a margin of 0.0009–0.0019 absolute miss ratio – and lost in the other 7. It beat MRU in every non-degenerate cell (8/8) and beat LFU more often than not (5 wins, 3 losses). Excluding wiki2018 widens rather than narrows the learned-vs-LRU/ SIEVE/random gap (macro miss ratio 0.7036 for learned vs. 0.6397 LRU, 0.6371 SIEVE, 0.6536 random-mean on the remaining 8 cells), so the comparison is not an artifact of averaging in the degenerate cells. In the 7 loss cells the gap is often large and not confined to the random-seed mean: at twemcache/cap128, learned (miss ratio 0.3909) loses to every one of the 20 random seeds (range 0.1586–0.1709), not just their mean.

Reading this result. This shows that, for this one frozen model under this held-out protocol, offline predictive structure on the finite-horizon counterfactual target did not translate into consistent closed-loop improvement over strong non-learned baselines. It does not, by itself, identify which mechanism is responsible: the offline selection criterion (validation-set regret/MAE on `y_loss`) is evaluated per-decision against a fixed history, while the closed-loop outcome is path-dependent on every eviction the policy itself previously made, so predicting the label well is a different task from inducing, through a sequence of choices, a cache-state trajectory with fewer total misses. Tie-degenerate supervision, action-underdetermined labels, continuation-policy mismatch, and this predict-then-act distribution shift are all consistent with the observed gap; none is established as the cause by this evidence alone, and this paper does not attribute one.

Table 10: Offline $\leftrightarrow$ closed-loop correspondence and continuation-policy robustness, summarized. The continuation rows use Set C (discriminative-under-LRU). Sampled random evidence is available only at capacities 32/128; full-population evidence is MRU-only.

<i>Offline$\leftrightarrow$closed-loop, $H=16$ ($n=10$ cells; SIEVE excluded)</i>		
metric	MRU-vs-LRU	random-vs-LRU
Concordant / discordant / tied (of 10)	7/1/2	6/2/2
Pearson r	0.835	0.755
Spearman ρ	0.915	0.829
Kendall τ -b	0.773	0.591
<i>Sampled continuation robustness, Set C, by capacity</i>		
metric	cap. 32	cap. 128
MRU median / mean Jaccard	1.000 / 0.9958	1.000 / 0.9999
MRU mean cross-cont. regret	0.0018	0.0001
MRU strict reversal fraction	2.7×10^{-5}	0.0
random median / mean Jaccard	1.000 / 0.7251	1.000 / 0.8522
random mean cross-cont. regret	0.0463	0.0197
random strict reversal fraction	1.0×10^{-3}	1.5×10^{-6}
<i>Full-population MRU census, Set C</i>		
metric	all capacities	new capacities 64/256
Scope	764,282 decision-H units	391,499 decision-H units
median / mean Jaccard	1.000 / 0.998459	1.000 / 0.999425
mean cross-cont. regret	0.000638	0.000267
strict reversal fraction	5.38×10^{-7}	2.83×10^{-7}
LRU-opt. still MRU-opt. fraction	0.999396	0.999745
robustness classification	ROBUST	ROBUST

16. Discussion

What LAFC-Evict enables. A reusable, candidate-level, finite-horizon counterfactual supervision surface, materialized once from real traces and usable across value-regression, best-candidate-selection, and pairwise formulations (Sections 2–8) without every study re-deriving its own labels from raw traces (Section 18 situates this against prior learned-cache work, most of which derives labels internally and non-reusably).

What the evaluation reveals, beyond the surface itself. Four findings that only emerge from evaluating the surface against closed-loop behavior, not from describing the surface alone: the target’s discriminativeness is workload-structured, not incidental (Section 9); that structure is explained, not just observed, by a single trace-level property, capacity-scale reuse (Section 12); offline and closed-loop policy orderings agree in most, not all, informative regimes, with the exceptions identified rather than smoothed over (Section 11); and the canonical LRU-continuation assumption is robust, with quantified limits, at the capacities tested (Section 13).

Why degenerate regimes are useful. wiki2018’s complete degeneracy and alibaba-block/cap32’s near-zero offline signal are explained results (Section 12), not unexplained anomalies. Mechanistically understood degenerate cases make the non-degenerate claims more trustworthy and answer the earlier criticism that tie-heaviness was stated but not investigated.

Offline surrogate evaluation vs. closed-loop performance. This paper’s position, supported by Sections 11 and 10, is that the offline↔closed-loop correspondence itself – not the offline surrogate alone – is the contribution: the two exceptions found were identified by the same symmetric procedure that found the 8 agreeing cells, not selected for balance, and are characterized rather than smoothed over (Section 14 gives the resulting practitioner guidance in detail).

Continuation robustness as currently validated. The robustness finding in Section 13 matters because it addresses a specific threat to the label’s validity: that the canonical LRU continuation, not the eviction choice itself, is what actually drives the reported target ordering. Within its tested scope (Section 14 gives the exact capacities and numbers), it is not: switching continuation policy leaves the ordering essentially unchanged. This is a statement about the tested alternatives, not a claim that every stateful deployed continuation policy would preserve the same ordering.

What the frozen learned-policy check adds. Section 15 connects

this paper’s larger message to an actual trained model: offline supervision can be highly structured (Section 9) and even correspond to closed-loop behavior in most non-learned-policy regimes (Section 11), yet a model selected on that supervision’s own offline criterion did not translate into consistent closed-loop improvement over strong non-learned baselines. Predicting a per-decision label well and inducing a good cache-state trajectory through a sequence of self-caused actions are different objects, and family heterogeneity persists in this check exactly as it does throughout the paper (Section 10). This is evidence about one frozen model under one protocol, not a general claim that learned eviction cannot work.

Methodological limits are collected in Section 17. For this discussion, the load-bearing ones are target discriminativeness, small- n linkage evidence, and the tested scope of continuation policies.

17. Limitations, Ethics, and Reuse

This evaluation surface has several important constraints, and the practical guidance in Section 14 should be read together with them rather than instead of them.

First, the main label is a finite-horizon, continuation-policy-specific counterfactual miss cost, not an offline-optimal label for the full future control problem. Good performance on this label does not automatically imply on-line deployment gains, which is exactly why Sections 10 and 11 evaluate closed-loop behavior directly rather than relying on the offline label alone.

Second, horizon-dependent results have two scopes. The reusable release uses $H \in \{4, 8, 16\}$. The extended sensitivity experiment compares $H \in \{16, 32, 64, 128\}$ using existing H16 evidence and completed long-horizon Wulver outputs, but it does not change the public release contents. H16 is the canonical H16-eligible population, while H32–H128 use the long-horizon-eligible production denominator; this is why paired family $\times$ capacity deltas are the safest basis for interpreting within-cell horizon trends.

Third, the canonical labels use LRU continuation after a forced eviction. The sampled MRU/random study and full-population MRU census in Section 13 support robustness only within their stated scopes; random continuation is not validated at population scale, and no experiment covers every deployment policy.

Fourth, the target is substantially tied. Across the full audited canonical dataset, the all-tied fraction is approximately 0.68 and the unique-winner

fraction is exactly 0.0 – no audited decision in this dataset has a single uniquely-best candidate victim. Discriminateness varies strongly by family, capacity, and horizon; one family, wiki2018, is a fully degenerate negative control at every capacity and horizon tested (Section 12 gives the causal explanation).

Fifth, the evaluated dataset uses unit-object, count-based capacities (32, 64, 128, or 256 objects) with unconditional admission of the requested object. Variable object sizes, byte-weighted miss cost, latency-aware cost, and dirty-write cost are not modeled by this label family, and admission is never withheld. Claims in this paper should be read as scoped to this abstraction and not generalized to byte-addressable or admission-gated cache models without further evidence.

Sixth, closed-loop evidence itself has two family-specific caveats that must be carried alongside any result that uses it. `metacdn`’s closed-loop evidence is validation-window evidence, not a temporally held-out test window (`twem-cache`’s is test-window). `metakv` is the only family whose scored window begins at an empty cache (a cold start, identifiable from the identity evictions = misses - capacity); this is plausibly, but not conclusively, connected to `metakv` being the one Tier-1 cell where LRU falls to third place, behind both SIEVE and random-mean (Section 12).

Seventh, a known data-generation issue remains open: in the audited canonical data, the predictor-victim flag is identical to the LRU-victim flag throughout, and related predictor, bucket, and confidence fields include constant or placeholder values. This paper does not make any claim that depends on those predictor fields being informative.

Eighth, benchmark reuse must preserve provenance distinctions. The release exposes generated supervision and derived views, but it does not erase the attribution, citation, or licensing requirements of the upstream raw traces from which candidate rows were derived; those raw traces remain external artifacts with their own governance constraints. The open subset is also intentionally narrower than the full generated corpus for redistribution reasons, and not every selected family appears in every split of the evaluated release package. The exact generator commit for the full historical release corpus is not completely reconstructed; this is orthogonal to the audited evidence reported in this paper but is disclosed here for reproducibility completeness.

Finally, this paper reports closed-loop results for LRU, MRU, random, and SIEVE under the Tier-1 protocol (Section 10), and, separately, a frozen learned-policy closed-loop check under an independently constructed held-

out protocol (Section 15). That check covers one model class, one feature set, one target ($H = 16$ `y_loss`), and two capacities; it found no consistent closed-loop improvement over LRU, SIEVE, or random, and should be read as a single confirmatory result, not an exhaustive study of learned-policy architectures, targets, or tuning, and not a general claim about learned cache eviction beyond this frozen configuration.

18. Related Work

18.1. Infrastructure, Traces, and Benchmarks

Cache studies rely on realistic traces and reusable tooling. The UMass Trace Repository and production workload studies provide realistic traces for trace-driven evaluation [14, 15, 13]. CacheLib, libCacheSim, and `cache_dataset` provide cache-engine context and simulation support for the same purpose [16, 17, 18]. Broader systems artifacts such as YCSB, OLTP-Bench, Join Order Benchmark, and Park show the value of reusable evaluation infrastructure for systems and learning-augmented systems research [19, 20, 21, 22]. Most directly, Cache-Coliseum is a NeurIPS 2025 benchmark for evaluating learning-augmented caching algorithms, with multiple algorithm implementations and predictors [23, 24]. It benchmarks algorithms against each other; LAFC-Evict instead supplies the labels an algorithm would train or evaluate against – reusable per-decision, per-candidate finite-horizon counterfactual supervision, plus decision and pairwise derived views (full positioning in Section 18.5).

18.2. Learned and Adaptive Cache Replacement

Recent learned-caching work typically contributes a policy, policy-selection scheme, or learning framework; the systems below differ in model class, target signal, and deployment setting, but all study how to make a cache choose better online or with lower overhead. LRB, LHD, GL-Cache, and 3L-Cache are one such cluster [7, 25, 9, 11]. Within this cluster, 3L-Cache specifically targets the computational-overhead barrier to deploying a learned per-object eviction policy, using bidirectional sampling and parameter auto-tuning to cut CPU cost while remaining a per-object policy rather than a reusable supervision artifact [11]. LeCaR, CACHEUS, MAT, and ALPS are another [6, 10, 26, 27]. HR-Cache, Parrot, Hawkeye, and Mockingjay round out this group [28, 8, 29, 30]. HALP [12] takes the same

general approach and is discussed further below for its similarity to LAFC-Evict’s pairwise view. HALP is especially close to LAFC-Evict’s pairwise view because it trains from preference comparisons between candidate evictions in production, but its signal is generated and consumed online by a deployed policy. LAFC-Evict’s pairwise view is instead a static reformatting of already-materialized finite-horizon counterfactual labels. Baleen addresses the adjacent admission/prefetching axis rather than per-candidate eviction labels [31].

18.3. *Heuristics and Learning-Augmented Caching*

Classical and modern policies define the policy landscape that LAFC-Evict interfaces with. Belady’s look-ahead optimum is the offline-optimal reference point against which all of the below are measured [32]. LIRS, ARC, CLOCK-Pro, and GreedyDual-Size are classical recency/frequency-aware policies [2, 1, 33, 34]. TinyLFU, W-TinyLFU, and S3-FIFO are more recent lightweight, admission-aware designs [35, 36, 4]. Lazy-promotion/quick-demotion, SIEVE, and LAH/S4-FIFO round out this group, each emphasizing a different robustness/overhead tradeoff [37, 3, 5]. LAH departs from per-object scoring by instead learning the cache-level parameters of a static heuristic (S3-FIFO, as S4-FIFO) with an asynchronous control-plane/data-plane split; LAFC-Evict does not propose a replacement policy or controller at either granularity, and instead exposes the counterfactual supervision such controllers could be trained or evaluated against [5]. Learning-augmented caching theory instead studies algorithms with predictions and robustness/consistency guarantees [38, 39, 40, 41]. Recent work in this line narrows the robustness gap for learning-augmented paging to within an additive constant of the optimal competitive ratio [42]; LAFC-Evict does not supply such a guarantee – it instead asks how reusable per-decision supervision for eviction choices can be constructed and characterized, a complementary rather than competing question. The subsampling study of Wang et al. identifies extremely long decision and reward horizons as a practical obstacle for reinforcement-learning cache policies and uses subsampling to reduce that horizon [43]; this motivates treating temporal horizon as a first-class difficulty, but does not validate LAFC-Evict’s particular $H \in \{4, 8, 16\}$ choices.

18.4. *Offline versus Deployed Evaluation*

Offline and deployed behavior need not agree when actions affect future observations. DAgger formalizes this distribution-shift problem for imitation learning and structured prediction; LAFC-Evict does not implement DAgger, but its closed-loop checks are motivated by the same caution [44]. In recommender systems, Castells and Moffat similarly survey why offline evaluation should not be equated with online utility [45]. Within caching, Qiu, Yang, and Harchol-Balter show a cache-local metric can even move opposite to end-to-end performance: beyond a point, higher hit ratio can reduce throughput because it changes hit-path contention [46]. This paper asks a narrower question – whether finite-horizon counterfactual miss-cost gaps align with closed-loop miss-ratio gaps – and reports a bounded, regime-characterized answer.

18.5. *Positioning LAFC-Evict*

Given this literature, LAFC-Evict’s contribution is a public cache-eviction artifact that releases reusable candidate-level finite-horizon counterfactual supervision, together with decision-level and pairwise task views, at production scale and decoupled from any particular downstream learned method. It complements traces, simulators, Cache-Coliseum-style algorithm benchmarks, learned/adaptive policies, and learning-augmented theory by supplying a reusable label surface for offline value regression, best-candidate selection, preference learning, and label characterization – not a new online eviction policy, and not a claim to be the first use of future-access supervision, preference learning, or benchmarking in this space (scope stated once here rather than repeated per subsection above).

19. Conclusion

This paper positions LAFC-Evict as a benchmark for learned cache eviction rather than as a new policy paper. The core contribution is a release-oriented package built around candidate-level counterfactual supervision, derived decision and pairwise tasks, and validation-aware construction. The open release couples manifest-backed scale and end-to-end validation with summaries of candidate-label distributions, a lightweight linear value-regression baseline, a derived best-candidate selector, and pairwise-sample-backed feature baselines that show measurable decision-relevant signal in the released candidate features.

Returning to the five questions posed in Section 1: the offline target is discriminative in a workload-, capacity-, and horizon-dependent way rather than uniformly, with one trace family serving as a mechanistically explained negative control (RQ1). Offline candidate rankings and closed-loop cache performance agree in most workload/capacity regimes where the offline signal is not near zero, with two specific, characterized exceptions (RQ2). Capacity-scale reuse in the underlying trace correlates with both offline discriminativeness and closed-loop policy separation, most strongly at the longest of the three horizons in the reusable release ($H = 16$ of $H \in \{4, 8, 16\}$). The extended $H \in \{16, 32, 64, 128\}$ sensitivity check shows diminishing changes by H64–H128 while preserving the main conclusion that the target remains substantially tied even at H128 (RQ3, exploratory, $n = 10$ for the closed-loop linkage). The canonical LRU-continuation labels are not an artifact of that specific continuation choice within the tested scope (RQ4, detailed below). Taken together, these findings support concrete, scope-limited guidance on when the offline signal can be trusted in practice (RQ5, Section 14). A separate, frozen learned-policy closed-loop check (Section 15) puts that guidance to a direct test: the model did not consistently out-perform LRU, SIEVE, or random under its own held-out protocol, which is itself informative – offline supervision quality on this target does not automatically translate into closed-loop gains for this frozen configuration.

Those results warrant conservative interpretation. The linear regression baseline is weak, the best-candidate selector is a reproducible benchmark floor rather than deployment evidence, the pairwise results remain backed by the shipped capped sample rather than any full quadratic pairwise universe, and the labels remain finite-horizon counterfactuals rather than globally optimal online targets. The validated continuation studies substantially reduce concern that the canonical LRU continuation alone drives the reported target ordering, but only within the tested scope: sampled MRU/random at capacities 32/128 and population MRU at capacities 32/64/128/256. The frozen learned-policy check is similarly a single confirmatory point, not an exhaustive study of learned-policy design space; it does not license re-tuning against the same held-out cells used to obtain it. Future extensions can instead focus on alternative training targets, tie-aware objectives, other learned architectures evaluated on fresh held-out data, policies designed explicitly for the closed-loop state distribution a policy induces, online policy studies that sit above the benchmark layer, broader continuation policies, and broader public hosting where redistribution constraints permit it.

Acknowledgments

The author gratefully acknowledges Prof. Ioannis (Yiannis) Koutis for his guidance and support. Computationally intensive experiments used the Wulver high-performance computing system at the New Jersey Institute of Technology. The author also acknowledges support from the Google Cloud Research Credits Program; Cohere Labs through the Catalyst Grant Program, which provided \$1,000 in API credits; and CloudRift AI through the AI Builder Grant, which provided 1,000 credits. The author further thanks Amazon Web Services, through the AWS Open Data Sponsorship Program, for sponsored hosting of up to 600 GB for two years to support public access to LAFC-Evict.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The LAFC-Evict repository is public at <https://github.com/SoroushVahidi/lafc-evict-dataset>. Code in the repository is released under the MIT License; the dataset itself is released under CC0-1.0 per its CITATION.cff dataset-metadata record. Upstream trace governance is separate from both: alibaba-block, metacdn, metakv, and twemcache remain uncleared for public derived-data redistribution under their own source licensing/usage terms, independent of this repository's own code and dataset licenses, so they are represented here through validated summaries rather than a public download of their generated rows. The current public derived-data snapshot (v0.3) is wiki2018-only and is distributed through Hugging Face (<https://huggingface.co/datasets/SoroushVahidi/lafc-evict>) and AWS Open Data. Zenodo currently archives v0.2 only (version DOI 10.5281/zenodo.21895844; concept DOI 10.5281/zenodo.21895843); no v0.3 Zenodo version DOI has been minted.

Funding

This work received computing/API support through the Google Cloud Research Credits Program, the Cohere Labs Catalyst Grant Program (\$1,000 in API credits), and the CloudRift AI Builder Grant (1,000 credits). Public hosting of the LAFC-Evict dataset is supported by the AWS Open Data Sponsorship Program, which provides sponsored hosting of up to 600 GB for two years.

CRedit authorship contribution statement

Soroush Vahidi: Conceptualization, Methodology, Software, Formal analysis, Investigation, Data curation, Writing – original draft, Writing – review & editing, Visualization.

Appendix A. Additional Release Tables

This appendix carries release/schema-descriptive tables that support reproducibility but do not themselves answer a research question posed in the main text. Content is unchanged from the underlying release manifest; nothing here was recomputed for this appendix.

Table A.12: Decision-view-backed benchmark composition by family, capacity, horizon, and split.

dimension	counts
trace_family	wiki2018=598,560; alibaba-block=576,219; metakv=454,053; twemcache=397,494; metacdn=336,960
capacity	32=613,314; 64=599,172; 128=582,678; 256=568,122
horizon	4=787,762; 8=787,762; 16=787,762
split	test=242,418; train=1,746,882; val=373,986

Table A.13: Decision-view-backed candidate-count statistics. These values summarize the number of candidates per decision in the preserved release.

group	min	med	mean	max
overall	32	64	117.63	256
alibaba-block	32	64	118.35	256
metacdn	32	64	117.12	256
metakv	32	64	119.52	256
twemcache	32	64	111.52	256
wiki2018	32	64	119.85	256

Table A.14: Candidate-label and best-candidate-selection summaries from validated result records.

statistic	value
candidate rows summarized	277,995,072
y_loss mean / std	7.7561 / 4.6542
y_loss q05 / q25 / q50 / q75 / q95	2 / 4 / 7 / 11 / 16
y_value mean / std	-7.7561 / 4.6542
y_value q05 / q25 / q50 / q75 / q95	-16 / -11 / -7 / -4 / -2
manifest-backed decision rows	2,363,286
best-candidate evaluator decision count	2,363,286 (exact match)
best-candidate optimal-selection rate	0.8753
best-candidate regret mean / median / max	0.1251 / 0 / 3
random-selector expected optimal-selection rate	0.9912
random-selector expected regret	0.0088

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During manuscript preparation, the author used ChatGPT (OpenAI), Gemini (Google), Claude (Anthropic), and Perplexity AI for drafting and editing support, research organization, and literature/web exploration. After using these tools, the author reviewed and edited the content and takes full responsibility for the published article. No AI tool is listed as an author or contributor.

AI-assisted code and research workflows. ChatGPT, OpenAI Codex, Claude, Cohere services/models, and CloudRift-hosted AI models were also used where applicable for code development, debugging, experiment organization, and research workflows. Exact Cohere and CloudRift model identifiers were not preserved and are therefore not invented here. All scientific claims, code, tables, and figures were reviewed by the author and trace to validated repository artifacts rather than to AI output alone.

References

- [1] N. Megiddo, D. S. Modha, Arc: A self-tuning, low overhead replacement cache, in: Proceedings of the 2nd USENIX Conference on File and Storage Technologies (FAST), 2003, pp. 115–130.
URL <https://www.usenix.org/conference/fast-03/arc-self-tuning-low-overhead-replacement-cache>
- [2] S. Jiang, X. Zhang, Lirs: An efficient low inter-reference recency set replacement policy to improve buffer cache performance, in: Proceedings of the ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems, 2002, pp. 31–42. doi:10.1145/511334.511340.
- [3] Y. Zhang, J. Yang, Y. Yue, Y. Vigfusson, K. V. Rashmi, SIEVE is simpler than LRU: an efficient Turn-Key eviction algorithm for web caches, in: 21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24), USENIX Association, 2024, pp. 1229–1246.
URL <https://www.usenix.org/conference/nsdi24/presentation/zhang-yazhuo>
- [4] J. Yang, Y. Zhang, Z. Qiu, Y. Yue, K. V. Rashmi, Fifo queues are all you need for cache eviction, in: ACM SIGOPS 29th

Symposium on Operating Systems Principles, SOSP '23, 2023.
doi:10.1145/3600006.3613147.

URL https://www.pdl.cmu.edu/PDL-FTP/Storage/FIFOqueues-SOSP23_abs.shtml

- [5] H. Xia, W. Nixon, B. D. Marthen, P. Bhandari, J. Yang, Learning-augmented heuristics: Simple, yet smart, robust and interpretable cache eviction, in: 20th USENIX Symposium on Operating Systems Design and Implementation (OSDI 26), USENIX Association, 2026.
URL <https://www.usenix.org/conference/osdi26/presentation/xia>
- [6] G. Vietri, L. V. Rodriguez, W. A. Martinez, S. Lyons, J. Liu, R. Rangaswami, M. Zhao, G. Narasimhan, Driving cache replacement with ml-based lecar, in: Proceedings of the USENIX Workshop on Hot Topics in Storage Systems (HotStorage), 2018.
URL <https://www.usenix.org/conference/hotstorage18/presentation/vietri>
- [7] Z. Song, D. S. Berger, K. Li, W. Lloyd, Learning relaxed belady for content distribution network caching, in: 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20), USENIX Association, Santa Clara, CA, 2020, pp. 529–544.
URL <https://www.usenix.org/conference/nsdi20/presentation/song>
- [8] E. Liu, M. Hashemi, K. Swersky, P. Ranganathan, J. Ahn, An imitation learning approach for cache replacement, in: Proceedings of the 37th International Conference on Machine Learning, Vol. 119 of Proceedings of Machine Learning Research, PMLR, 2020, pp. 6237–6247.
URL <https://proceedings.mlr.press/v119/liu20f.html>
- [9] J. Yang, Z. Mao, Y. Yue, K. V. Rashmi, GL-Cache: Group-level learning for efficient and high-performance caching, in: 21st USENIX Conference on File and Storage Technologies (FAST 23), USENIX Association, Santa Clara, CA, 2023, pp. 115–134.
URL <https://www.usenix.org/conference/fast23/presentation/yang-juncheng>

- [10] L. V. Rodriguez, F. Yusuf, S. Lyons, E. Paz, R. Rangaswami, J. Liu, M. Zhao, G. Narasimhan, Learning cache replacement with cacheus, in: 19th USENIX Conference on File and Storage Technologies (FAST 21), USENIX Association, 2021, pp. 341–354.
URL <https://www.usenix.org/conference/fast21/presentation/rodriguez>
- [11] W. Zhou, Z. Niu, Y. Xiong, J. Fang, Q. Wang, 3L-Cache: Low overhead and precise learning-based eviction policy for caches, in: 23rd USENIX Conference on File and Storage Technologies (FAST 25), USENIX Association, Santa Clara, CA, 2025, pp. 237–254.
URL <https://www.usenix.org/conference/fast25/presentation/zhou-wenbin>
- [12] Z. Song, K. Chen, N. Sarda, D. Altınbüken, E. Brevdo, J. Coleman, X. Ju, P. Jurczyk, R. Schooler, R. Gummadi, HALP: Heuristic aided learned preference eviction policy for YouTube content delivery network, in: 20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23), USENIX Association, 2023, pp. 1149–1163.
URL <https://www.usenix.org/conference/nsdi23/presentation/song-zhenyu>
- [13] J. Yang, Y. Yue, K. V. Rashmi, A large-scale analysis of hundreds of in-memory cache clusters at twitter, in: 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20), 2020, pp. 191–208.
URL <https://www.usenix.org/conference/osdi20/presentation/yang>
- [14] University of Massachusetts Amherst, Umass trace repository, official trace repository website (2025).
URL <https://traces.cs.umass.edu/>
- [15] B. Atikoglu, Y. Xu, E. Frachtenberg, S. Jiang, M. Paleczny, Workload analysis of a large-scale key-value store, in: Proceedings of the ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems, 2012, pp. 53–64.
- [16] B. Berg, D. S. Berger, S. McAllister, I. Grosof, S. Gunasekar, J. Lu, M. Uhlar, J. Carrig, N. Beckmann, M. Harchol-Balter, G. R. Ganger,

- The CacheLib caching engine: Design and experiences at scale, in: 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20), USENIX Association, 2020, pp. 753–768.
URL <https://www.usenix.org/conference/osdi20/presentation/berg>
- [17] cacheMon, libCacheSim: A high performance cache simulator and library, <https://github.com/cacheMon/libCacheSim>, accessed 2026-07-03 (2026).
 - [18] cacheMon, A comprehensive open-source cache trace dataset, https://github.com/cacheMon/cache_dataset, accessed 2026-07-03 (2026).
 - [19] B. F. Cooper, A. Silberstein, E. Tam, R. Ramakrishnan, R. Sears, Benchmarking cloud serving systems with ycsb, in: Proceedings of the ACM Symposium on Cloud Computing, 2010, pp. 143–154.
 - [20] D. E. Difallah, A. Pavlo, C. Curino, P. Cudré-Mauroux, Oltp-bench: An extensible testbed for benchmarking relational databases, Proceedings of the VLDB Endowment 7 (4) (2013) 277–288.
 - [21] V. Leis, B. Radke, A. Gubichev, A. Mirchev, P. Boncz, A. Kemper, T. Neumann, Query optimization through the looking glass, and what we found running the join order benchmark, The VLDB Journal 27 (5) (2018) 643–668. doi:10.1007/s00778-017-0480-7.
 - [22] H. Mao, P. Negi, A. Narayan, H. Wang, J. Yang, H. Wang, R. Marcus, M. Khani, H. He, V. Nathan, F. Cangialosi, S. B. Venkatakrisnan, S. Weng, S. Han, T. Kraska, M. Alizadeh, Park: An open platform for learning-augmented computer systems, in: Advances in Neural Information Processing Systems 32, 2019.
URL https://papers.nips.cc/paper_files/paper/2019/hash/5a83f15f2b94ea59206c4312bb9c3d8f-Abstract.html
 - [23] P. Chen, H. Zhao, J. Zhang, X. Tang, Y. Wang, S. Deng, Robustifying learning-augmented caching efficiently without compromising 1-consistency, in: Advances in Neural Information Processing Systems 38, 2025. doi:10.52202/085713-4092.
URL https://papers.nips.cc/paper_files/paper/2025/hash/b1e7f61f40d68b2177857bfcbb195a507-Abstract-Conference.html

- [24] OptiSys-ZJU, Cache-Coliseum: A benchmark for learning-augmented caching algorithms, <https://github.com/OptiSys-ZJU/Cache-Coliseum>, accessed 2026-09-15 (2025).
- [25] N. Beckmann, H. Chen, A. Cidon, LHD: Improving cache hit rate by maximizing hit density, in: 15th USENIX Symposium on Networked Systems Design and Implementation (NSDI 18), USENIX Association, Renton, WA, 2018, pp. 389–403.
URL <https://www.usenix.org/conference/nsdi18/presentation/beckmann>
- [26] D. Yang, D. S. Berger, K. Li, W. Lloyd, A learned cache eviction framework with minimal overhead (2023). [arXiv:2301.11886](https://arxiv.org/abs/2301.11886), doi:10.48550/arXiv.2301.11886.
URL <https://arxiv.org/abs/2301.11886>
- [27] R. Shahout, R. Friedman, Sketching the path to efficiency: Lightweight learned cache replacement, in: 27th International Conference on Principles of Distributed Systems (OPODIS 2023), Vol. 286 of Leibniz International Proceedings in Informatics (LIPIcs), 2024, pp. 34:1–34:21. doi:10.4230/LIPIcs.OPODIS.2023.34.
- [28] H. Torabi, H. Khazaei, M. Litoiu, A learning-based caching mechanism for edge content delivery, in: Proceedings of the 15th ACM/SPEC International Conference on Performance Engineering, 2024, pp. 236–246. doi:10.1145/3629526.3645037.
- [29] A. Jain, C. Lin, Back to the future: Leveraging belady’s algorithm for improved cache replacement, in: Proceedings of the 43rd International Symposium on Computer Architecture, 2016, pp. 78–89. doi:10.1145/3007787.3001146.
- [30] I. Shah, A. Jain, C. Lin, Effective mimicry of belady’s min policy, in: 2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA), 2022, pp. 558–572. doi:10.1109/HPCA53966.2022.00048.
- [31] D. L.-K. Wong, H. Wu, C. Molder, S. Gunasekar, J. Lu, S. Khandkar, A. Sharma, D. S. Berger, N. Beckmann, G. R. Ganger, Baleen: ML admission & prefetching for flash caches, in: 22nd USENIX Conference on

- File and Storage Technologies (FAST 24), USENIX Association, 2024, pp. 347–371.
URL <https://www.usenix.org/conference/fast24/presentation/wong>
- [32] L. A. Belady, A study of replacement algorithms for a virtual-storage computer, *IBM Systems Journal* 5 (2) (1966) 78–101. doi:10.1147/sj.52.0078.
 - [33] S. Jiang, F. Chen, X. Zhang, Clock-pro: An effective improvement of the clock replacement, in: *Proceedings of the Annual USENIX Technical Conference*, 2005, pp. 323–336.
URL <https://www.usenix.org/conference/2005-usenix-annual-technical-conference/clock-pro-effective-improvement-clock-replacement>
 - [34] P. Cao, S. Irani, Cost-aware www proxy caching algorithms, in: *Proceedings of the USENIX Symposium on Internet Technologies and Systems*, 1997, pp. 193–206.
URL <https://www.usenix.org/conference/usits-97/cost-aware-www-proxy-caching-algorithms>
 - [35] G. Einziger, R. Friedman, B. Manes, Tinylfu: A highly efficient cache admission policy, *ACM Transactions on Storage* 13 (4) (2017) 35:1–35:31. doi:10.1145/3149371.
 - [36] G. Einziger, O. Eytan, R. Friedman, B. Manes, Adaptive software cache management, in: *Proceedings of the 19th International Middleware Conference*, 2018, pp. 94–106. doi:10.1145/3274808.3274816.
 - [37] J. Yang, Z. Qiu, Y. Zhang, Y. Yue, K. V. Rashmi, FIFO can be better than LRU: The power of lazy promotion and quick demotion, in: *19th Workshop on Hot Topics in Operating Systems (HotOS 23)*, ACM, 2023. doi:10.1145/3593856.3595887.
 - [38] T. Lykouris, S. Vassilvitskii, Competitive caching with machine learned advice, *Journal of the ACM* 68 (4) (2021) 24:1–24:25. doi:10.1145/3447579.
 - [39] A. Wei, Better and simpler learning-augmented online caching, in: *Approximation, Randomization, and Combinatorial Optimization*.

- Algorithms and Techniques (APPROX/RANDOM 2020), Vol. 176 of Leibniz International Proceedings in Informatics (LIPIcs), Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2020, pp. 60:1–60:17. doi:10.4230/LIPIcs.APPROX/RANDOM.2020.60.
URL <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.APPROX/RANDOM.2020.60>
- [40] J. Chłedowski, A. Polak, B. Szabucki, K. T. Żołna, Robust learning-augmented caching: An experimental study, in: Proceedings of the 38th International Conference on Machine Learning, Vol. 139 of Proceedings of Machine Learning Research, PMLR, 2021, pp. 1920–1930.
URL <https://proceedings.mlr.press/v139/chledowski21a.html>
 - [41] S. Im, R. Kumar, A. Petety, M. Purohit, Parsimonious learning-augmented caching, in: Proceedings of the 39th International Conference on Machine Learning, Vol. 162 of Proceedings of Machine Learning Research, PMLR, 2022, pp. 9588–9601.
URL <https://proceedings.mlr.press/v162/im22a.html>
 - [42] P. Chen, H. Zhao, X. Tang, Y. Wang, S. Deng, Towards optimal robustness in learning-augmented paging (2026). arXiv:2606.01342, doi:10.48550/arXiv.2606.01342.
URL <https://arxiv.org/abs/2606.01342>
 - [43] H. Wang, H. He, M. Alizadeh, H. Mao, Learning caching policies with subsampling, in: NeurIPS 2019 Workshop on Machine Learning for Systems, 2019.
URL https://mlforsystems.org/assets/papers/neurips2019/learning_wang_2019.pdf
 - [44] S. Ross, G. Gordon, D. Bagnell, A reduction of imitation learning and structured prediction to no-regret online learning, in: Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics, Vol. 15 of Proceedings of Machine Learning Research, PMLR, 2011, pp. 627–635.
URL <https://proceedings.mlr.press/v15/ross11a.html>
 - [45] P. Castells, A. Moffat, Offline recommender system evaluation: Challenges and new directions, AI Magazine 43 (2) (2022) 225–238. doi:10.1002/aaai.12051.

- [46] Z. Qiu, J. Yang, M. Harchol-Balter, Why increasing the hit ratio can hurt cache throughput, *Queueing Systems* 110 (2) (2026). doi:10.1007/s11134-026-09986-1.